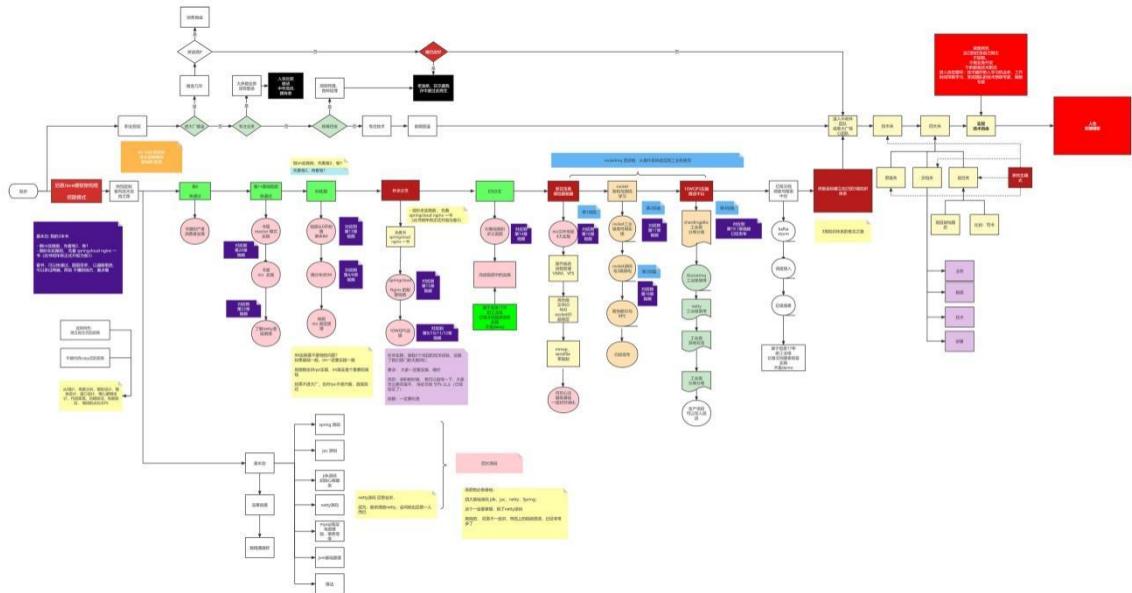


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

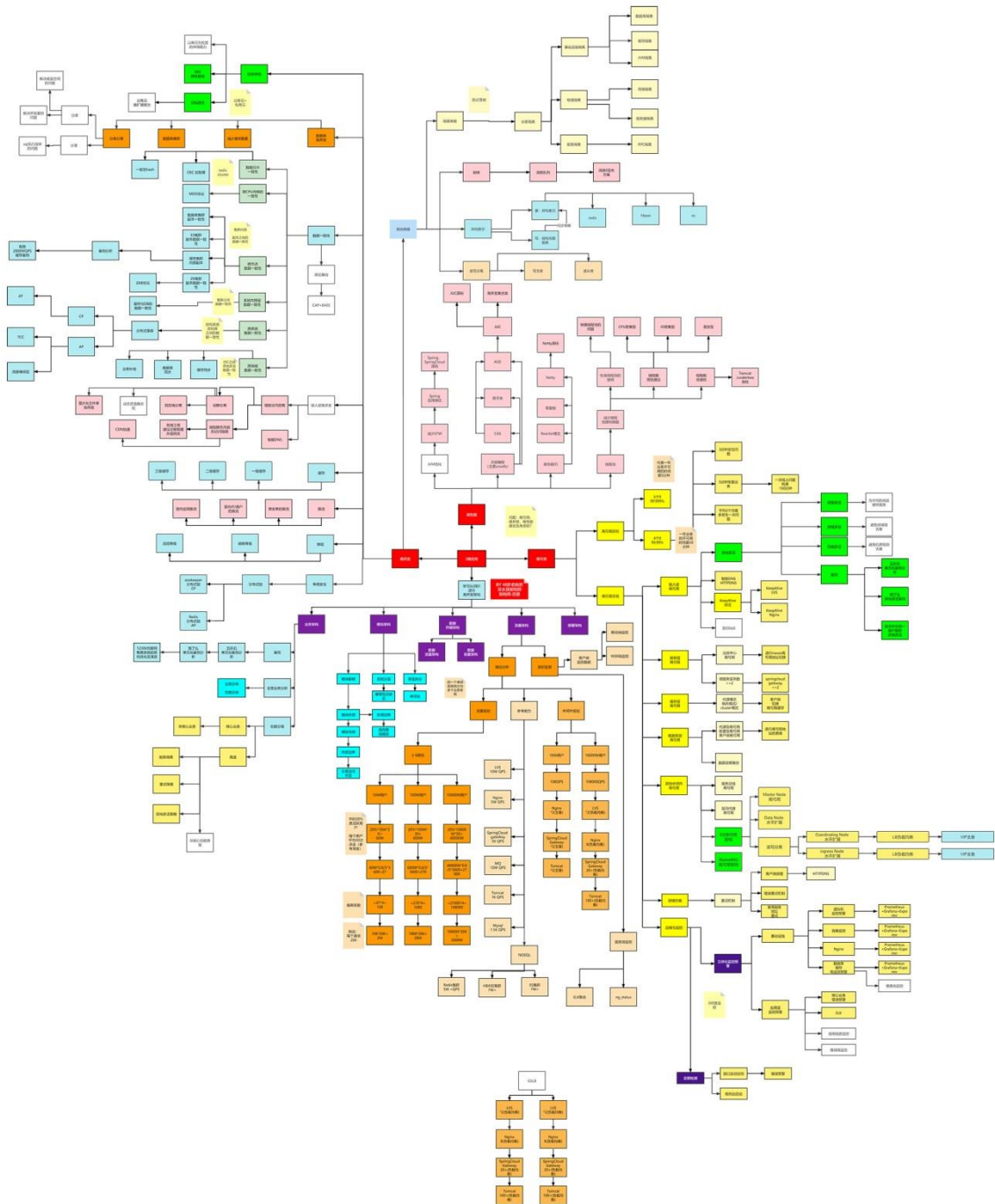
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

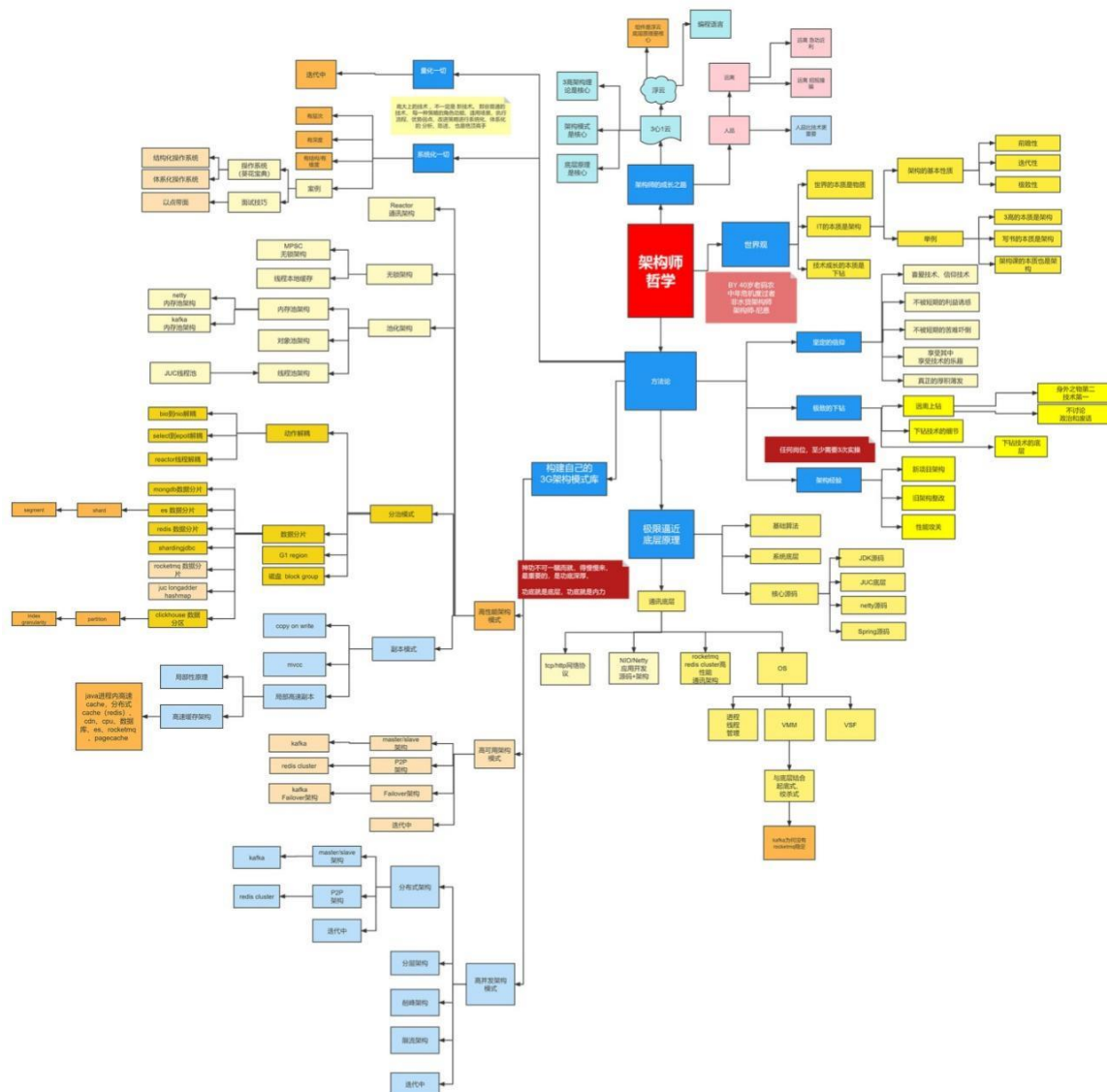
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

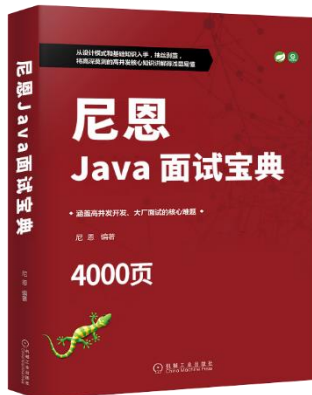
链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



尼恩Java面试宝典

40 个专题（卷王专供+ 史上最全 + 2023 面试必备）

详情：<https://www.cnblogs.com/crazymakercircle/p/13917138.html>



名称

- ❏ 专题01: JVM面试题 (卷王专供 + 史上最全 + 2022面试必备) -V81-from-尼恩Java面试宝典.pdf
- ❏ 专题02: Java算法面试题 (卷王专供 + 史上最全 + 2022面试必备) -V80-from-Java面试红宝书.pdf
- ❏ 专题03: Java基础面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题04: 架构设计面试题 (卷王专供+ 史上最全 + 2023面试必备) -V86-from-尼恩Java面试宝典.pdf
- ❏ 专题05: Spring面试题_专题06: SpringMVC_专题07: Tomcat面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩面试宝典-release.pdf
- ❏ 专题08: SpringBoot面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题09: 网络协议面试题 (卷王专供+ 史上最全 + 2023面试必备) -V46-from-尼恩Java面试宝典-release.pdf
- ❏ 专题10: TCP/IP协议 (卷王专供+ 史上最全 + 2022面试必备) -V57-from-Java面试红宝书.pdf
- ❏ 专题11: JUC并发包与容器类 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题12: 设计模式面试题 (卷王专供+ 史上最全 + 2022面试必备) -V84-from-Java面试红宝书.pdf
- ❏ 专题13: 死锁面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题14: Redis 面试题 (卷王专供+ 史上最全 + 2022面试必备) -V65-from-Java面试红宝书.pdf
- ❏ 专题15: 分布式锁 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题16: Zookeeper 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题17: 分布式事务面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题18: 一致性协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题19: Zab协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题20: Paxos 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题21: raft 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题22: Linux面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题23: Mysql 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V82-from-尼恩Java面试宝典.pdf
- ❏ 专题24: SpringCloud 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V12-from-Java面试红宝书-release.pdf
- ❏ 专题25: Netty 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题26: 消息队列面试题: RabbitMQ、Kafka、RocketMQ (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf
- ❏ 专题27: 内存泄漏 内存溢出 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题28: JVM 内存溢出 实战 (卷王专供+ 史上最全 + 2023面试必备) -V17-from-Java面试红宝书-release.pdf
- ❏ 专题29: 多线程面试题 (卷王专供+ 史上最全 + 2023面试必备) -V66-from-Java面试红宝书.pdf
- ❏ 专题30: HR面试题: 过五关斩六将后, 小心阴沟翻船! (史上最全、避坑宝典) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题31: Hash/链表面试题 (卷王专供+ 史上最全 + 2022面试必备) -V68-from-Java面试红宝书.pdf
- ❏ 专题32: 大厂面试的基本流程和面试准备 (阿里、腾讯、网易、京东、头条.....) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题33: BST、AVL、RB红黑树、三大核心数据结构 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- ❏ 专题34: Elasticsearch面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-Java面试红宝书-release.pdf
- ❏ 专题35: Mybatis面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩Java面试宝典-release.pdf
- ❏ 专题36: Dubbo面试题 (卷王专供+ 史上最全 + 2023面试必备) -V21-from-尼恩Java面试宝典-release.pdf
- ❏ 专题37: Docker面试题 (卷王专供+ 史上最全 + 2023面试必备) -V47-from-尼恩Java面试宝典.pdf
- ❏ 专题38: K8S面试题 (卷王专供+ 史上最全 + 2023面试必备) -V59-from-尼恩Java面试宝典.pdf
- ❏ 专题39: Nginx面试题 (卷王专供+ 史上最全 + 2023面试必备) -V27-from-尼恩Java面试宝典-release.pdf
- ❏ 专题40: 操作系统面试题 (卷王专供+ 史上最全 + 2023面试必备) -V28-from-尼恩Java面试宝典-release.pdf
- ❏ 专题41: 大厂面试真题 (卷王专供+ 史上最全 + 2023面试必备) -V84-from-尼恩Java面试宝典.pdf

未来职业，如何突围：三栖架构师

未来职业，如何突围？

技术自由圈



——未来超级架构师社区

领路式指导

FSAC 三栖合一架构师

Future Super Architect Community

- 第一栖：Java 架构
- 第二栖：GO 架构
- 第三栖：大数据 架构

尼恩JAVA硬核架构班

会员制

提供技术方向指导，
职业生涯指导，少坑坑，少弯路

简历指导

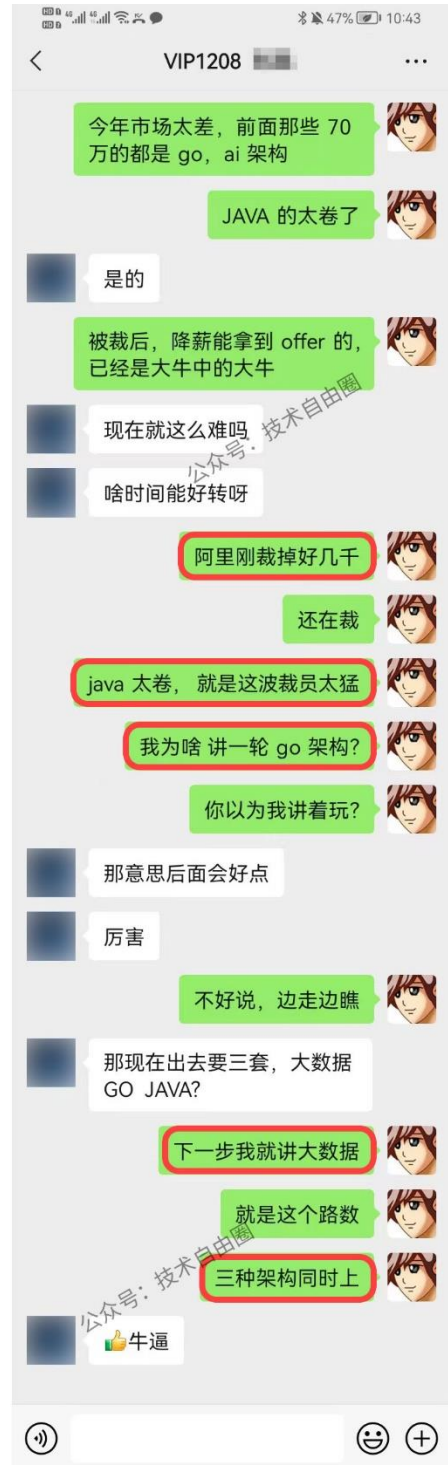
有助成功就业、跳槽大厂
挪窝涨薪必备

实操性

项目都是老架构师
在生产上实操过的项目

非水货

老架构师，不是水货架构师
《Java高并发三部曲》为证



专题25：Netty面试题（史上最全、定期更新）

本文版本说明：V88

此文的格式，由markdown 通过程序转成而来，由于很多表格，没有来的及调整，出现一个格式问题，尼恩在此给大家道歉啦。

由于社群很多小伙伴，在面试，不断的交流最新的面试难题，所以，《[尼恩Java面试宝典](#)》，后面会不断升级，迭代。

本专题，作为 《尼恩Java面试宝典》专题之一，《尼恩Java面试宝典》一共**41个面试专题**，后续还会增加

版本升级记录

升级说明：

V88升级说明（2023-07-23）：

美团二面：epoll性能那么高，为什么？

V3升级说明（2022-05-22）：一图搞懂netty

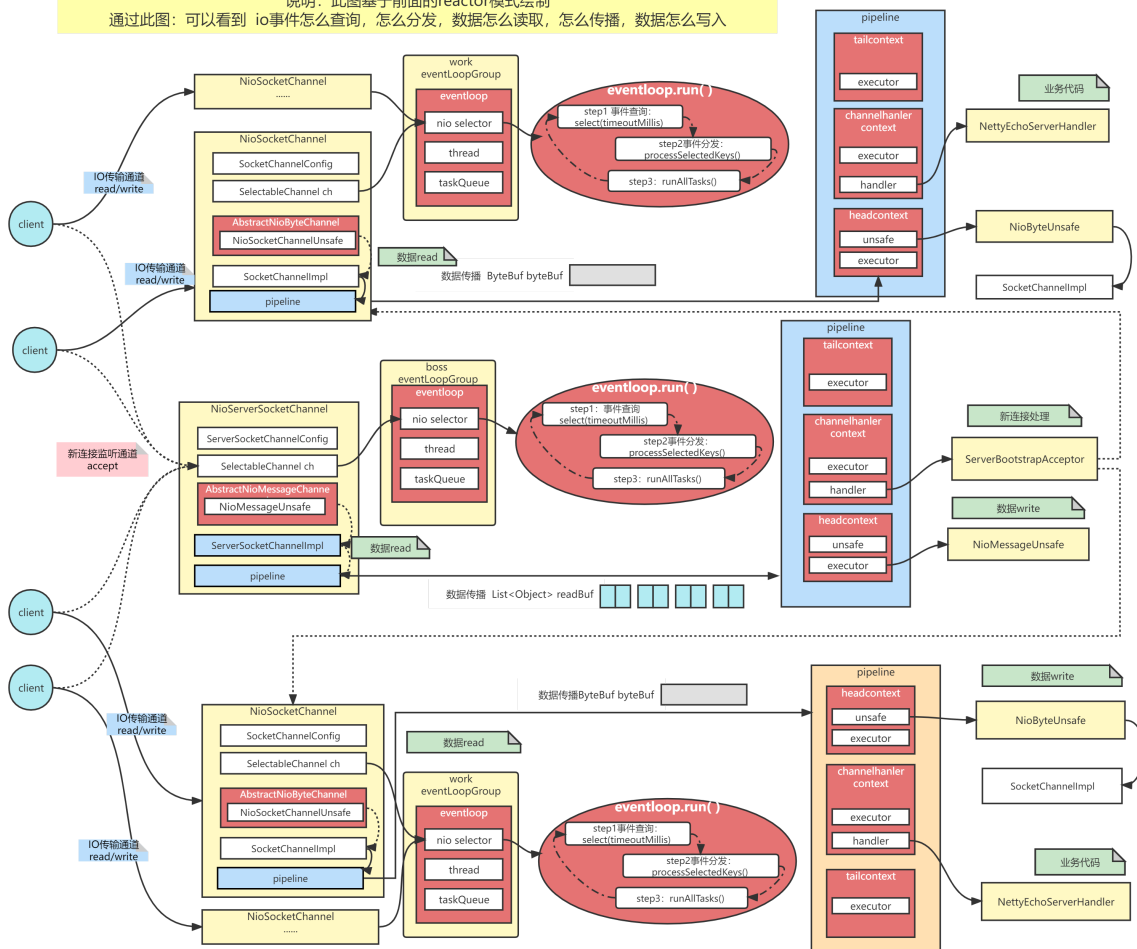
Netty很难，一直以来，没有一张图能比较深入的介绍清楚netty

于是，**尼恩绘制了一张:Netty架构图**

通过此图，应该对Netty的核心组件，有一个清晰的了解

这个图上都有：**io事件怎么查询，怎么分发，数据怎么读取，数据怎么传播，数据怎么写入**

说明：此图基于前面的reactor模式绘制
通过此图：可以看到 io事件怎么查询，怎么分发，数据怎么读取，怎么传播，数据怎么写入



关于此图的链接：可以参考 疯狂创客圈 微信群的 历史记录

《尼恩面试宝典》升级的规划：

后续基本上，**每一个月，都会发布一次**，最新版本，可以扫描扫架构师尼恩微信，发送“领取电子书”获取。

尼恩的微信二维码在哪里呢？请参见文末

面试问题交流说明：

如果遇到面试难题，或者职业发展问题，或者中年危机问题，都可以来 疯狂创客圈 社群交流，

加入交流群，加尼恩微信即可，

入交流群，加尼恩微信即可，发送“入群”



史上最全 Java 面试题之：Netty 篇

40岁老架构师尼恩暗语：

如果在简历上写了Netty，那么：下面的面试题，最好都会

如果要面试高端的开发、大厂的开发、或者架构师，那么： 简历上一定要写netty

so，下面的面试题，越烂熟于心，越好

Netty 是什么？

Netty是一个异步事件驱动的网络应用程序框架，用于快速开发可维护的高性能协议服务器和客户端。

Netty是基于nio的，它封装了jdk的nio，让我们使用起来更加方法灵活。

聊聊：JDK原生NIO程序的问题？

JDK原生也有一套网络应用程序API，但是存在一系列问题，主要如下：

- NIO的类库和API繁杂，使用麻烦，你需要熟练掌握Selector、ServerSocketChannel、SocketChannel、ByteBuffer等
- 需要具备其它的额外技能做铺垫，例如熟悉Java多线程编程，因为NIO编程涉及到Reactor模式，你必须对多线程和网络编程非常熟悉，才能编写出高质量的NIO程序
- 可靠性能力补齐，开发工作量和难度都非常大。

例如客户端面临断连重连、网络闪断、半包读写、失败缓存、网络拥塞和异常码流的处理等等，NIO编程的特点是功能开发相对容易，但是可靠性能力补齐工作量和难度都非常大

- JDK NIO的BUG，例如臭名昭著的select bug，它会导致Selector空轮询，最终导致CPU 100%。

官方声称在JDK1.6版本的update18修复了该问题，但是直到JDK1.7版本该问题仍旧存在，只不过该bug发生概率降低了一些而已，它并没有被根本解决

Netty 的优势有哪些？

- 使用简单：封装了 NIO 的很多细节，使用更简单。
- 功能强大：预置了多种编解码功能，支持多种主流协议。
- 定制能力强：可以通过 ChannelHandler 对通信框架进行灵活地扩展。
- 性能高：通过与其他业界主流的 NIO 框架对比，Netty 的综合性能最优。
- 稳定：Netty 修复了已经发现的所有 NIO 的 bug，让开发人员可以专注于业务本身。
- 社区活跃：Netty 是活跃的开源项目，版本迭代周期短，bug 修复速度快。

Netty 的应用场景有哪些？

典型的应用有：

- 阿里分布式服务框架 Dubbo，默认使用 Netty 作为基础通信组件，
- 还有 RocketMQ 也是使用 Netty 作为通讯的基础。

Netty常见的使用场景如下：

- 互联网行业 在分布式系统中，各个节点之间需要远程服务调用，高性能的RPC框架必不可少，Netty作为异步高新能的通信框架,往往作为基础通信组件被这些RPC框架使用。典型的应用有：阿里分布式服务框架Dubbo的RPC框架使用Dubbo协议进行节点间通信，Dubbo协议默认使用Netty作为基础通信组件，用于实现各进程节点之间的内部通信。
- 游戏行业 无论是手游服务端还是大型的网络游戏，Java语言得到了越来越广泛的应用。Netty作为高性能的基础通信组件，它本身提供了TCP/UDP和HTTP协议栈。非常方便定制和开发私有协议栈，账号登录服务器，地图服务器之间可以方便的通过Netty进行高性能的通信
- 大数据领域 经典的Hadoop的高性能通信和序列化组件Avro的RPC框架，默认采用Netty进行跨节点通信，它的Netty Service基于Netty框架二次封装实现

Netty的特点

Netty的对JDK自带的NIO的API进行封装，解决上述问题，主要特点有：

- 设计优雅 适用于各种传输类型的统一API - 阻塞和非阻塞Socket 基于灵活且可扩展的事件模型，可以清晰地分离关注点 高度可定制的线程模型 - 单线程，一个或多个线程池 真正的无连接数据报套接字支持（自3.1起）
- 高性能、高吞吐、低延迟、低消耗
- 最小化不必要的内存复制
- 安全 完整的SSL / TLS和StartTLS支持
- 高并发：Netty 是一款基于 NIO（Nonblocking IO，非阻塞IO）开发的网络通信框架，对比于 BIO（Blocking I/O，阻塞IO），他的并发性能得到了很大提高。
- 传输快：Netty 的传输依赖于零拷贝特性，尽量减少不必要的内存拷贝，实现了更高效率的传输。
- 封装好：Netty 封装了 NIO 操作的很多细节，提供了易于使用调用接口。
- 社区活跃，不断更新 社区活跃，版本迭代周期短，发现的BUG可以被及时修复，同时，更多的新功能会被加入
- 使用方便 详细记录的Javadoc，用户指南和示例 没有其他依赖项，JDK 5（Netty 3.x）或6（Netty 4.x）就足够了

Netty 高性能表现在哪些方面？

- IO 线程模型：通过多线程Reactor反应器模式，在应用层实现异步非阻塞（异步事件驱动）架构，用最少的资源做更多的事。
- 内存零拷贝：尽量减少不必要的内存拷贝，实现了更高效率的传输。
- 内存池设计：申请的内存可以重用，主要指直接内存。内部实现是用一颗二叉查找树管理内存分配情况。（**具体请参考尼恩稍后的手写内存池**）
- 对象池设计：Java对象可以重用，主要指Minor GC非常频繁的对象，如ByteBuffer。并且，**对象池使用无锁架构**，性能非常高。（**具体请参考尼恩稍后的手写对象池**）
- mpsc无锁编程：串行化处理读写，避免使用锁带来的性能开销。
- 高性能序列化协议：支持 protobuf 等高性能序列化协议。

聊聊：NIO的组成？

Buffer：与Channel进行交互，数据是从Channel读入缓冲区，从缓冲区写入Channel中的

flip方法：反转此缓冲区，将position给limit，然后将position置为0，其实就是切换读写模式

clear方法：清除此缓冲区，将position置为0，把capacity的值给limit。

rewind方法：重绕此缓冲区，将position置为0

DirectByteBuffer可减少一次系统空间到用户空间的拷贝。但Buffer创建和销毁的成本更高，不可控，通常会用内存池来提高性能。直接缓冲区主要分配给那些易受基础系统的本机I/O 操作影响的大型、持久的缓冲区。如果数据量比较小的中小应用情况下，可以考虑使用heapBuffer，由JVM进行管理。

Channel：表示 IO 源与目标打开的连接，是双向的，但不能直接访问数据，只能与Buffer 进行交互。通过源码可知，FileChannel的read方法和write方法都导致数据复制了两次！

Selector可使一个单独的线程管理多个Channel，open方法可创建Selector，register方法向多路复用器注册通道，可以监听的事件类型：读、写、连接、accept。注册事件后会产生一个SelectionKey：它表示SelectableChannel 和Selector 之间的注册关系，wakeup方法：使尚未返回的第一个选择操作立即返回，唤醒的

原因是：注册了新的channel或者事件；channel关闭，取消注册；优先级更高的事件触发（如定时器事件），希望及时处理。

Selector在Linux的实现类是EPollSelectorImpl，委托给EPollArrayWrapper实现，其中三个native方法是对epoll的封装，而EPollSelectorImpl.implRegister方法，通过调用epoll_ctl向epoll实例中注册事件，还将注册的文件描述符(fd)与SelectionKey的对应关系添加到fdToKey中，这个map维护了文件描述符与SelectionKey的映射。

fdToKey有时会变得非常大，因为注册到Selector上的Channel非常多（百万连接）；过期或失效的Channel没有及时关闭。fdToKey总是串行读取的，而读取是在select方法中进行的，该方法是非线程安全的。

Pipe：两个线程之间的单向数据连接，数据会被写到sink通道，从source通道读取

NIO的服务端建立过程：Selector.open()：打开一个Selector；ServerSocketChannel.open()：创建服务端的Channel；bind()：绑定到某个端口上。并配置非阻塞模式；register()：注册Channel和关注的事件到Selector上；select()轮询拿到已经就绪的事件

聊聊： BIO、NIO和AIO的区别？

BIO：一个连接一个线程，客户端有连接请求时服务器端就需要启动一个线程进行处理。线程开销大。伪异步IO：将请求连接放入线程池，一对多，但线程还是很宝贵的资源。

NIO：一个请求一个线程，但客户端发送的连接请求都会注册到多路复用器上，多路复用器轮询到连接有I/O请求时才启动一个线程进行处理。

AIO：一个有效请求一个线程，客户端的I/O请求都是由OS先完成了再通知服务器应用去启动线程进行处理，

BIO是面向流的，NIO是面向缓冲区的；

BIO的各种流是阻塞的。而NIO是非阻塞的；

BIO的Stream是单向的，而NIO的channel是双向的。

NIO的特点：事件驱动模型、单线程处理多任务、非阻塞I/O，I/O读写不再阻塞，而是返回0、基于block的传输比基于流的传输更高效、更高级的IO函数zero-copy、IO多路复用大大提高了Java网络应用的可伸缩性和实用性。基于Reactor线程模型。

在Reactor模式中，事件分发器等待某个事件或者可应用或个操作的状态发生，事件分发器就把这个事件传给事先注册的事件处理函数或者回调函数，由后者来做实际的读写操作。如在Reactor中实现读：注册读就绪事件和相应的事件处理器、事件分发器等待事件、事件到来，激活分发器，分发器调用事件对应的处理器、事件处理器完成实际的读操作，处理读到的数据，注册新的事件，然后返还控制权。

Netty 和 Tomcat 的区别？

- 作用不同：Tomcat 是 Servlet 容器，可以视为 Web 服务器，而 Netty 是异步事件驱动的网络应用程序框架和工具用于简化网络编程，例如TCP和UDP套接字服务器。
- 协议不同：Tomcat 是基于 http 协议的 Web 服务器，而 Netty 能通过编程自定义各种协议，因为 Netty 本身自己能编码/解码字节流，所有 Netty 可以实现，HTTP 服务器、FTP 服务器、UDP 服务器、RPC 服务器、WebSocket 服务器、Redis 的 Proxy 服务器、MySQL 的 Proxy 服务器等等。

聊聊：Netty是怎么实现高性能设计的？

Netty作为高性能IO组件的扛鼎制作，

高性能设计的核心：巧妙的结合高性能IO模型和线程模型，相得益彰，达到了高性能、高吞吐、低延迟、低消耗的目标

其I/O模型 高性能epoll/select 模型

其线程模型为多线程 reactor 反应器模型

I/O模型 决定如何收发数据，

线程模型 决定如何处理数据

相得益彰，在应用层达到了异步IO的效果。

聊聊：I/O模型

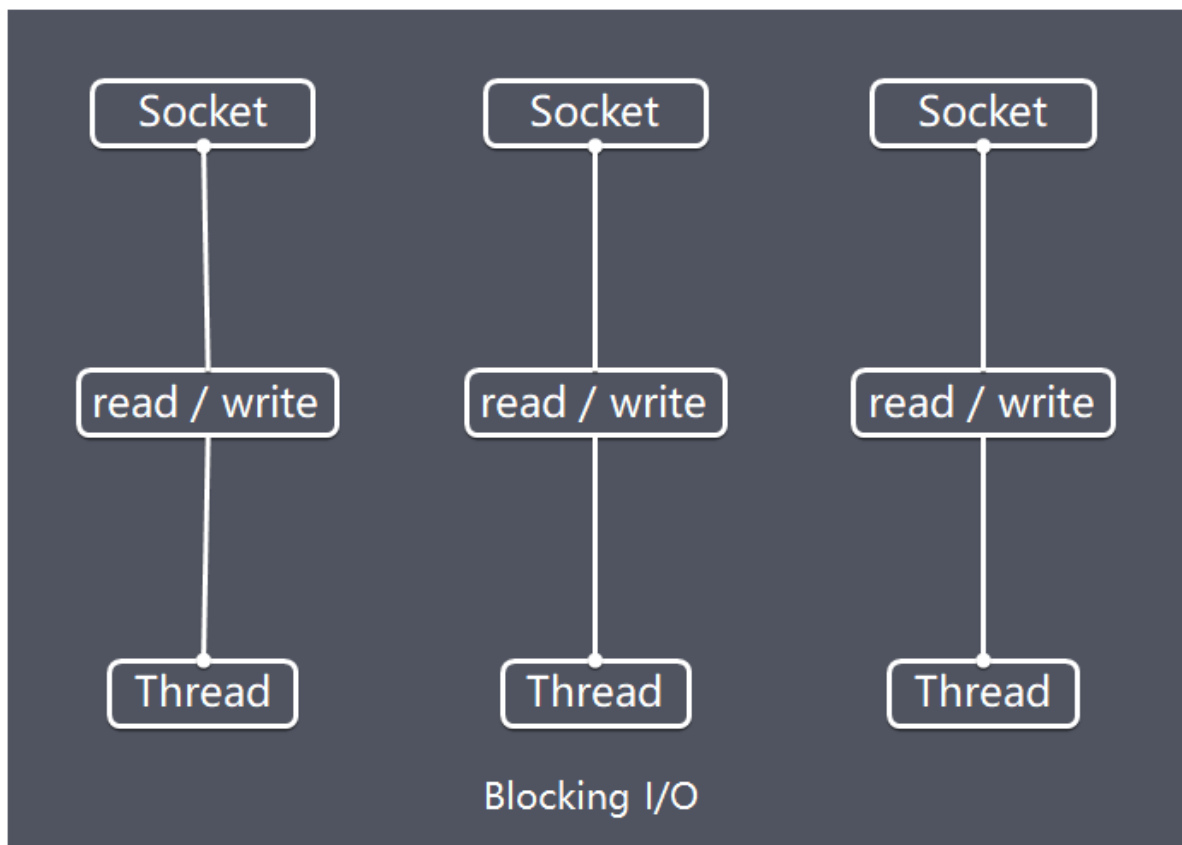
用什么样的通道将数据发送给对方，BIO、NIO或者AIO，I/O模型在很大程度上决定了框架的性能

答案请参考[《Java高并发核心编程 卷1》](#)

书中，对io模型介绍得非常系统，并且不断完善

阻塞I/O

传统阻塞型I/O(BIO)可以用下图表示：



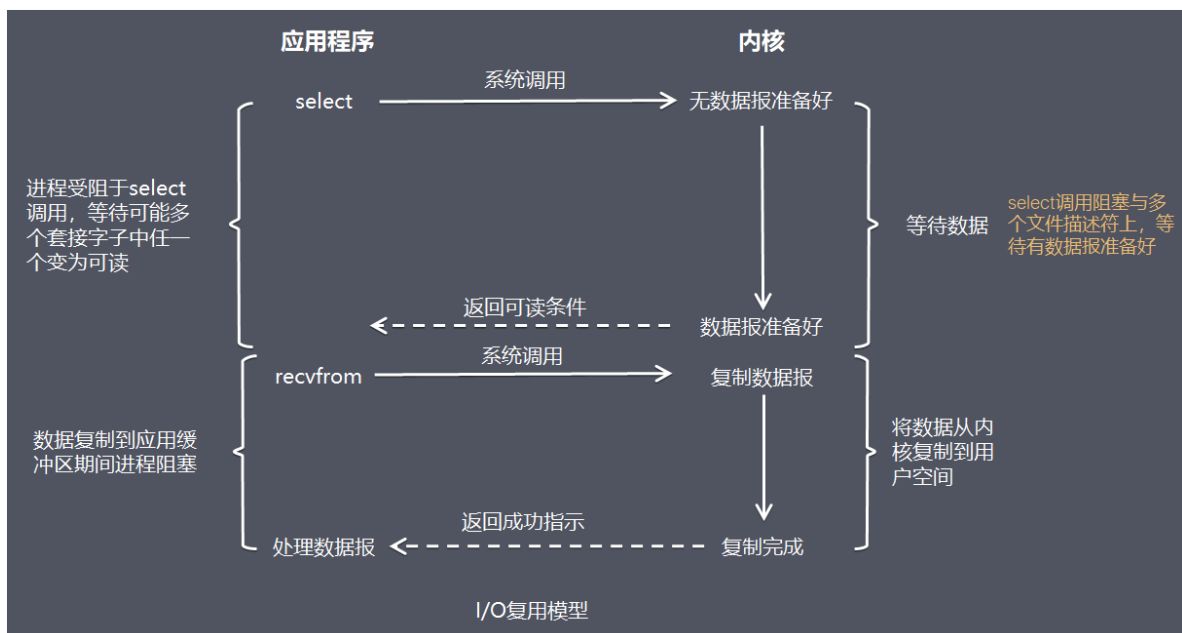
特点

- 每个请求都需要独立的线程完成数据read, 业务处理, 数据write的完整操作

问题

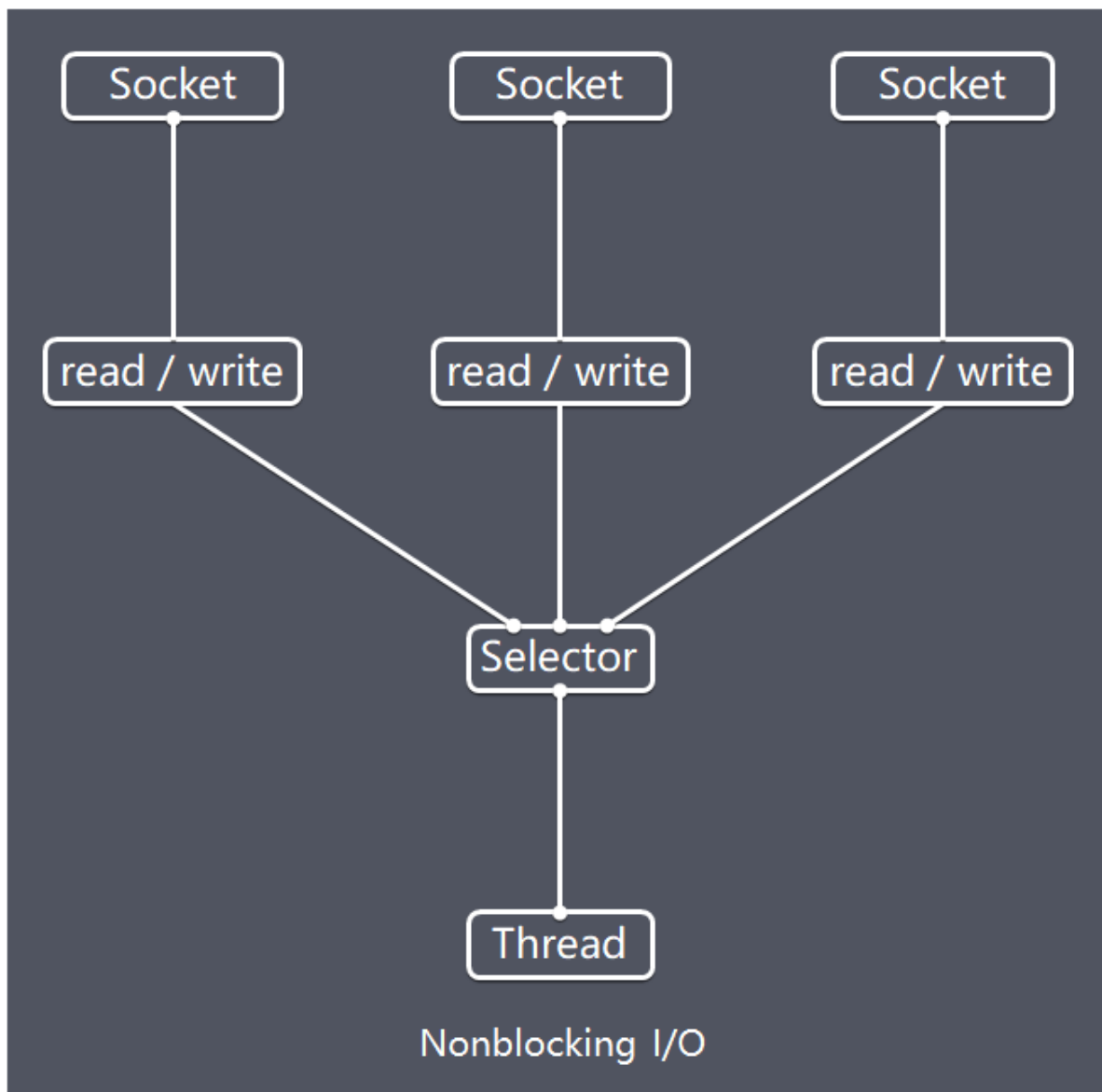
- 当并发数较大时，需要创建大量线程来处理连接，系统资源占用较大
- 连接建立后，如果当前线程暂时没有数据可读，则线程就阻塞在read操作上，造成线程资源浪费

I/O复用模型



在I/O复用模型中，会用到select，这个函数也会使进程阻塞，但是和阻塞I/O所不同的，这两个函数可以同时阻塞多个I/O操作，而且可以同时多个读操作，多个写操作的I/O函数进行检测，直到有数据可读或可写时，才真正调用I/O操作函数

Netty的非阻塞I/O的实现关键是基于I/O复用模型，这里用Selector对象表示：



Netty的IO线程NioEventLoop由于聚合了多路复用器Selector，可以同时并发处理成百上千个客户端连接。当线程从某客户端Socket通道进行读写数据时，若没有数据可用时，该线程可以进行其他任务。线程通常将非阻塞IO的空闲时间用于在其他通道上执行IO操作，所以单独的线程可以管理多个输入和输出通道。

由于读写操作都是非阻塞的，这就可以充分提升IO线程的运行效率，避免由于频繁IO阻塞导致的线程挂起，一个IO线程可以并发处理N个客户端连接和读写操作，这从根本上解决了传统同步阻塞IO—连接—线程模型，架构的性能、弹性伸缩能力和可靠性都得到了极大的提升。

基于buffer

传统的IO是面向字节流或字符流的，以流式的方式顺序地从一个Stream中读取一个或多个字节，因此也就不能随意改变读取指针的位置。

在NIO中，抛弃了传统的IO流，而是引入了Channel和Buffer的概念。在NIO中，只能从Channel中读取数据到Buffer中或将数据Buffer中写入到Channel。

基于buffer操作不像传统IO的顺序操作，NIO中可以随意地读取任意位置的数据

聊聊：AIO 是什么？

答案请参考[《Java高并发核心编程 卷1》](#)

书中，对io模型介绍得非常系统，并且不断完善

聊聊：NIO和BIO到底有什么区别？有什么关系？

1. NIO是以块的方式处理数据，BIO是以字节流或者字符流的形式去处理数据。
2. NIO是通过缓存区和通道的方式处理数据，BIO是通过InputStream和OutputStream流的方式处理数据。
3. NIO的通道是双向的，BIO流的方向只能是单向的。
4. NIO采用的多路复用的同步非阻塞IO模型，BIO采用的是普通的同步阻塞IO模型。
5. NIO的效率比BIO要高，NIO适用于网络IO，BIO适用于文件IO。

聊聊：NIO是如何实现同步非阻塞的？

一个线程 Thread 使用一个选择器Selector监听多个通道 Channel 上的IO事件，从而让一个线程就可以处理多个IO事件。通过配置监听的通道Channel为非阻塞，那么当Channel上的IO事件还未到达时，线程会在select方法被挂起，让出CPU资源。直到监听到Channel有IO事件发生时，才会进行相应的响应和处理。

Selector能够检测多个注册的通道上是否有IO事件发生(注意:多个 Channel 以事件的方式可以注册到同一个Selector)，如果有事件发生，便获取事件然后针对每个事件进行相应的处理。这样就可以只用一个单线程去管理多个通道，也就是管理多个连接和请求。

Selector只有在通道上有真正的IO事件发生时，才会进行相应的处理，这就不必为每个连接都创建一个线程，避免线程资源的浪费和多线程之间的上下文切换导致的开销。

聊聊：BIO和NIO应用场景

- 1、**BIO** 方式适用于连接数目比较小且固定的架构，这种方式对服务器资源要求比较高，并发局限于应用中，JDK1.4以前的唯一选择。
- 2、**NIO** 方式适用于连接数目多且连接比较短（轻操作）的架构，比如聊天服务器，弹幕系统，服务器间通讯等。JDK1.4 开始支持。

聊聊：阻塞/非阻塞区别

答案请参考[《Java高并发核心编程 卷1》](#)

书中，对io模型介绍得非常系统，并且不断完善

聊聊：同步/异步区别

答案请参考[《Java高并发核心编程 卷1》](#)

书中，对io模型介绍得非常系统，并且不断完善

聊聊：select、poll和epoll的区别

消息传递方式：

select：内核需要将消息传递到用户空间，需要内核的拷贝动作；

poll：同上；

epoll：通过内核和用户空间共享一块内存来实现，性能较高；

文件句柄剧增后带来的IO效率问题：

select：因为每次调用都会对连接进行线性遍历，所以随着FD剧增后会造成遍历速度的“线性下降”的性能问题；

poll：同上；

epoll：由于epoll是根据每个FD上的callable函数来实现的，只有活跃的socket才会主动调用callback，所以在活跃socket较少的情况下，使用epoll不会对性能产生线性下降的问题，如果所有socket都很活跃的情况下，可能会有性能问题；

支持一个进程所能打开的最大连接数：

select：单个进程所能打开的最大连接数，是由FD_SETSIZE宏定义的，其大小是32个整数大小（在32位的机器上，大小是3232,64位机器上FD_SETSIZE=3264），我们可以对其进行修改，然后重新编译内核，但是性能无法保证，需要做进一步测试；

poll：本质上与select没什么区别，但是他没有最大连接数限制，他是基于链表来存储的；

epoll：虽然连接数有上线，但是很大，1G内存的机器上可以打开10W左右的连接；

聊聊：主要的IO线程模型有哪些？

数据报如何读取？读取之后的编解码在哪个线程进行，编解码后的消息如何派发，线程模型的不同，对性能的影响也非常大。

Connection Per Thread（一个线程处理一个连接）线程模式

在Java的OIO编程中，最原始的网络服务器程序，一般是用一个while循环，不断地监听端口是否有新的连接。

如果有，那么就调用一个处理函数来完成传输处理，示例代码如下：

```

while(true){

    socket = accept(); //阻塞，接收连接

    handle(socket) ;    //读取数据、业务处理、写入结果

}

```

这种方法的最大问题是：如果前一个网络连接的handle (socket) 没有处理完，那么后面的新连接没法被服务端接收，于是后面的请求就会被阻塞住，这样就导致服务器的吞吐量太低。

这对于服务器来说，这是一个严重的问题。

为了解决这个严重的连接阻塞问题，出现了一个极为经典模式：Connection Per Thread（一个线程处理一个连接）模式。示例代码如下：

```

package com.crazymakercircle.idemo.OIO;
//...省略import导入的Java类
class ConnectionPerThread implements Runnable {
    public void run() {
        try {
            //服务器监听socket
            ServerSocket serverSocket =
                new ServerSocket(NioDemoConfig.SOCKET_SERVER_PORT);
            while (!Thread.interrupted()) {
                Socket socket = serverSocket.accept();
                //接收一个连接后，为socket连接，新建一个专属的处理器对象
                Handler handler = new Handler(socket);
                //创建新线程，专门负责一个连接的处理
                new Thread(handler).start();
            }

        } catch (IOException ex) { /* 处理异常 */ }
    }
    //处理器，这里将内容回显到客户端
    static class Handler implements Runnable {
        final Socket socket;
        Handler(Socket s) {
            socket = s;
        }
        public void run() {
            while (true) {
                try {
                    byte[] input = new byte[1024];
                    /* 读取数据 */
                    socket.getInputStream().read(input);
                    /* 处理业务逻辑，获取处理结果*/
                    byte[] output = null;
                    /* 写入结果 */
                    socket.getOutputStream().write(output);
                } catch (IOException ex) { /*处理异常*/ }
            }
        }
    }
}

```

```
    }  
    }  
}
```

以上示例代码中，对于每一个新的网络连接都分配给一个线程。每个线程都独自处理自己负责的socket连接的输入和输出。当然，服务器的监听线程也是独立的，任何的socket连接的输入和输出处理，不会阻塞到后面新socket连接的监听和建立，这样，服务器的吞吐量就得到了提升。早期版本的Tomcat服务器，就是这样实现的。

Connection Per Thread模式（一个线程处理一个连接）的优点是：解决了前面的新连接被严重阻塞的问题，在一定程度上，较大的提高了服务器的吞吐量。

Connection Per Thread模式的缺点是：对应于大量的连接，需要耗费大量的线程资源，对线程资源要求太高。在系统中，线程是比较昂贵的系统资源。如果线程的数量太多，系统无法承受。而且，线程的反复创建、销毁、线程的切换也需要代价。因此，在高并发的应用场景下，多线程OIO的缺陷是致命的。

新的问题来了：如何减少线程数量，比如说让一个线程同时负责处理多个socket连接的输入和输出，行不行呢？

看上去，没有什么不可以。但是，实际上作用不大。为什么呢？传统OIO编程中每一次socket传输的IO读写处理，都是阻塞的。在同一时刻，一个线程里只能处理一个socket的读写操作，前一个socket操作被阻塞了，其他连接的IO操作同样无法被并行处理。所以在OIO中，即使是一个线程同时负责处理多个socket连接的输入和输出，同一时刻，该线程也只能处理一个连接的IO操作。

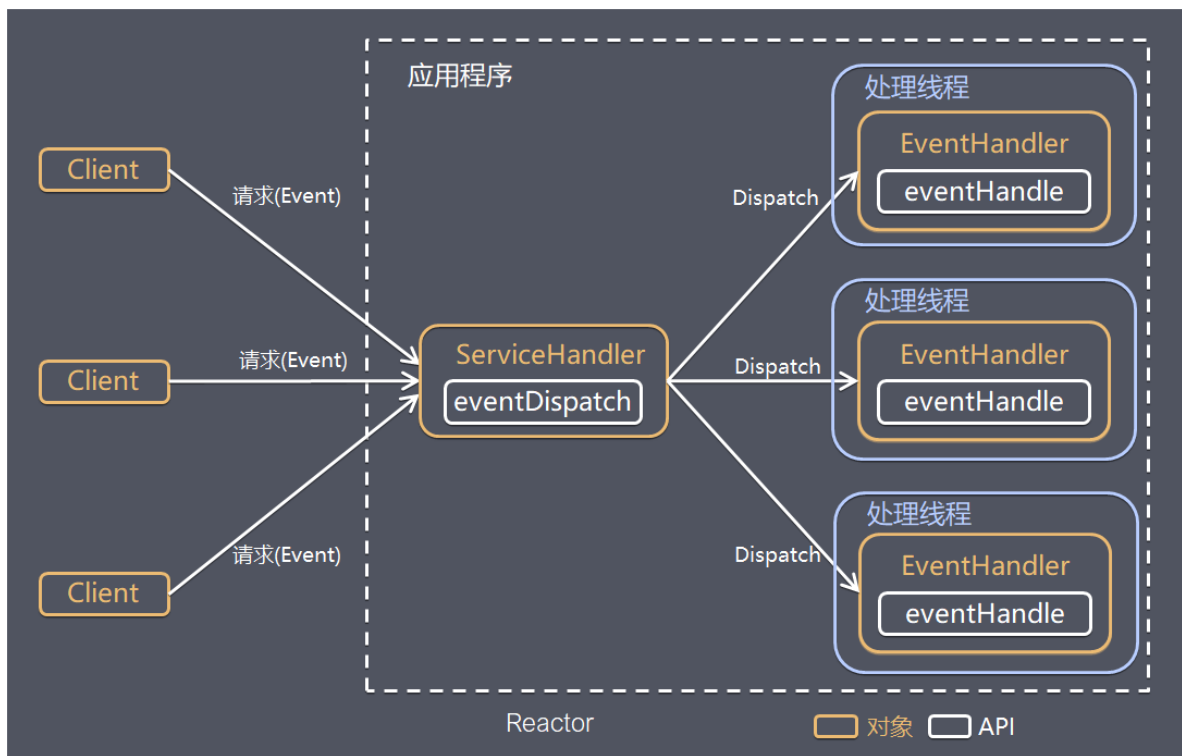
如何解决Connection Per Thread模式的巨大缺陷呢？一个有效途径是：使用Reactor反应器模式。用反应器模式对线程的数量进行控制，做到一个线程处理大量的连接。

Reactor线程模型

Reactor是反应堆/反应器的意思，Reactor模型，是指通过一个或多个输入同时传递给服务处理器的服务请求的**事件驱动处理模式**。服务端程序处理传入多路请求，并将它们同步分派给请求对应的处理线程，Reactor模式也叫Dispatcher模式，即I/O多了复用统一监听事件，收到事件后分发(Dispatch给某进程)，是编写高性能网络服务器的必备技术之一。

Reactor模型中有2个关键组成：

- Reactor Reactor在一个单独的线程中运行，负责监听和分发事件，分发给适当的处理程序来对IO事件做出反应。它就像公司的电话接线员，它接听来自客户的电话并将线路转移到适当的联系人
- Handlers 处理程序执行I/O事件要完成的实际事件，类似于客户想要与之交谈的公司中的实际官员。Reactor通过调度适当的处理程序来响应I/O事件，处理程序执行非阻塞操作



取决于Reactor的数量和Handler线程数量的不同，Reactor模型有3个变种

- 单Reactor单线程
- 单Reactor多线程
- 主从Reactor多线程

可以这样理解，Reactor就是一个执行while (true) { selector.select(); ...}循环的线程，会源源不断的产生新的事件，称作反应堆很贴切。

篇幅关系，这里不再具体展开Reactor特性、优缺点比较，有兴趣的读者可以参考

[《Java高并发核心编程 卷1》](#)

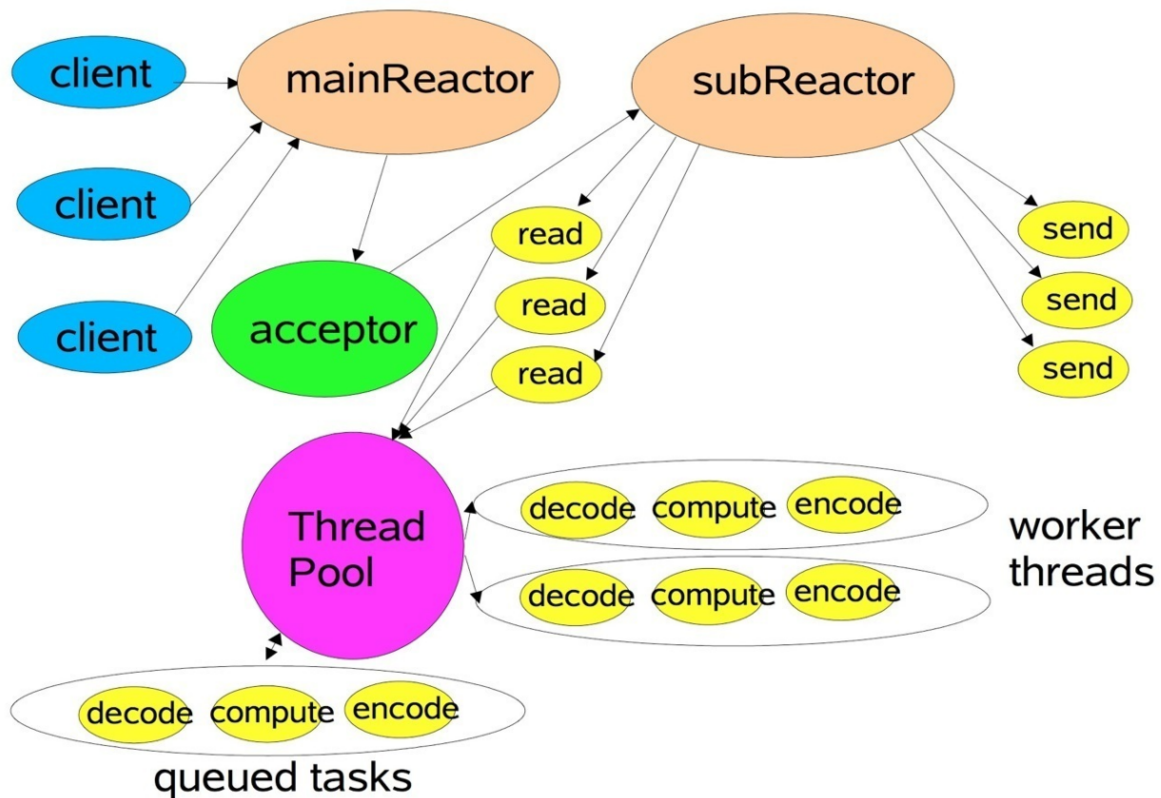
书中，对Reactor 模型介绍得非常系统，并且不断完善

Netty线程模型

Netty主要**基于主从Reactors多线程模型**（如下图）做了一定的修改，其中主从Reactor多线程模型有多个Reactor：MainReactor和SubReactor：

- MainReactor负责客户端的连接请求，并将请求转交给SubReactor
- SubReactor负责相应通道的IO读写请求
- 非IO请求（具体逻辑处理）的任务则会直接写入队列，等待worker threads进行处理

这里引用Doug Lee大神的Reactor介绍：[Scalable IO in Java](#)里面关于主从Reactor多线程模型的图



特别说明的是：

虽然Netty的线程模型基于主从Reactor多线程，借用了MainReactor和SubReactor的结构，但是实际实现上，SubReactor和Worker线程在同一个线程池中：

```

EventLoopGroup bossGroup = new NioEventLoopGroup();
EventLoopGroup workerGroup = new NioEventLoopGroup();
ServerBootstrap server = new ServerBootstrap();
server.group(bossGroup, workerGroup)
    .channel(NioServerSocketChannel.class)

```

上面代码中的bossGroup 和workerGroup是Bootstrap构造方法中传入的两个对象，这两个group均是线程池

- bossGroup线程池则只是在bind某个端口后，获得其中一个线程作为MainReactor，专门处理端口的accept事件，**每个端口对应一个boss线程**
- workerGroup线程池会被各个SubReactor和worker线程充分利用

异步处理

异步的概念和同步相对。当一个异步过程调用发出后，调用者不能立刻得到结果。实际处理这个调用的部件在完成后，通过状态、通知和回调来通知调用者。

Netty中的I/O操作是异步的，包括bind、write、connect等操作会简单的返回一个ChannelFuture，调用者并不能立刻获得结果，通过Future-Listener机制，用户可以方便的主动获取或者通过通知机制获得IO操作结果。

当future对象刚刚创建时，处于非完成状态，调用者可以通过返回的ChannelFuture来获取操作执行的状态，注册监听函数来执行完成后的操，常见有如下操作：

- 通过isDone方法来判断当前操作是否完成
- 通过isSuccess方法来判断已完成的当前操作是否成功

- 通过getCause方法来获取已完成的当前操作失败的原因
- 通过isCancelled方法来判断已完成的当前操作是否被取消
- 通过addListener方法来注册监听器，当操作已完成(isDone方法返回完成)，将会通知指定的监听器；如果future对象已完成，则理解通知指定的监听器

例如下面的的代码中绑定端口是异步操作，当绑定操作处理完，将会调用相应的监听器处理逻辑

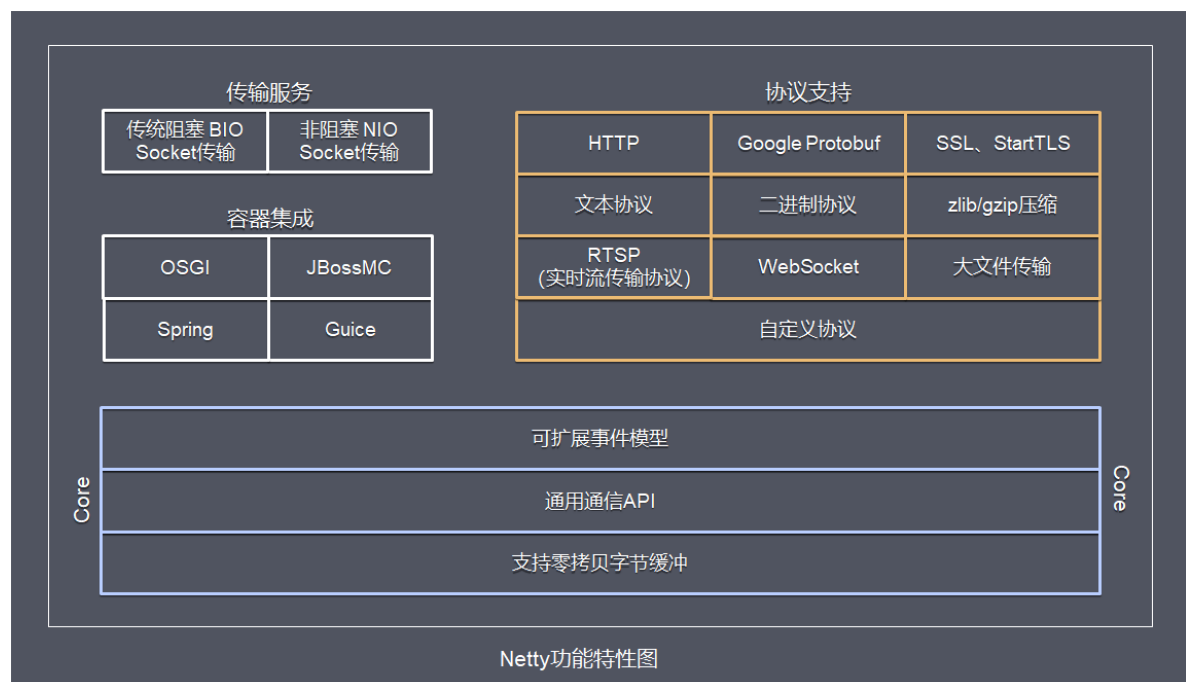
```
serverBootstrap.bind(port).addListener(future -> {
    if (future.isSuccess()) {
        System.out.println(new Date() + ": 端口[" + port + "]绑定成功!");
    } else {
        System.err.println("端口[" + port + "]绑定失败!");
    }
});
```

相比传统阻塞I/O，执行I/O操作后线程会被阻塞住，直到操作完成；异步处理的好处是不会造成线程阻塞，线程在I/O操作期间可以执行别的程序，在高并发情形下会更稳定和更高的吞吐量。

Netty架构设计

前面介绍完Netty相关一些理论介绍，下面从功能特性、模块组件、运作过程来介绍Netty的架构设计

功能特性



- 传输服务 支持BIO和NIO
- 容器集成 支持OSGI、JBossMC、Spring、Guice容器
- 协议支持 HTTP、Protobuf、二进制、文本、WebSocket等一系列常见协议都支持。还支持通过实行编码解码逻辑来实现自定义协议
- Core核心 可扩展事件模型、通用通信API、支持零拷贝的ByteBuf缓冲对象

模块组件

Bootstrap、ServerBootstrap

Bootstrap意思是引导，一个Netty应用通常由一个Bootstrap开始，主要作用是配置整个Netty程序，串联各个组件，Netty中Bootstrap类是客户端程序的启动引导类，ServerBootstrap是服务端启动引导类。

Future、ChannelFuture

正如前面介绍，在Netty中所有的IO操作都是异步的，不能立刻得知消息是否被正确处理，但是可以过一会等它执行完成或者直接注册一个监听，具体的实现就是通过Future和ChannelFutures，他们可以注册一个监听，当操作执行成功或失败时监听会自动触发注册的监听事件。

Channel

Netty网络通信的组件，能够用于执行网络I/O操作。Channel为用户提供：

- 当前网络连接的通道的状态（例如是否打开？是否已连接？）
- 网络连接的配置参数（例如接收缓冲区大小）
- 提供异步的网络I/O操作(如建立连接，读写，绑定端口)，异步调用意味着任何I/O调用都将立即返回，并且不保证在调用结束时所请求的I/O操作已完成。调用立即返回一个ChannelFuture实例，通过注册监听器到ChannelFuture上，可以I/O操作成功、失败或取消时回调通知调用方。
- 支持关联I/O操作与对应的处理程序

不同协议、不同的阻塞类型的连接都有不同的 Channel 类型与之对应，下面是一些常用的 Channel 类型

- NioSocketChannel，异步的客户端 TCP Socket 连接
- NioServerSocketChannel，异步的服务器端 TCP Socket 连接
- NioDatagramChannel，异步的 UDP 连接
- NioSctpChannel，异步的客户端 Sctp 连接
- NioSctpServerChannel，异步的 Sctp 服务器端连接 这些通道涵盖了 UDP 和 TCP网络 IO以及文件 IO。

Selector

Netty基于Selector对象实现I/O多路复用，通过 Selector, 一个线程可以监听多个连接的Channel事件，当向一个Selector中注册Channel 后，Selector 内部的机制就可以自动不断地查询(select) 这些注册的Channel是否有已就绪的I/O事件(例如可读, 可写, 网络连接完成等)，这样程序就可以很简单地使用一个线程高效地管理多个 Channel。

NioEventLoop

NioEventLoop中维护了一个线程和任务队列，支持异步提交执行任务，线程启动时会调用NioEventLoop的run方法，执行I/O任务和非I/O任务：

- I/O任务 即selectionKey中ready的事件，如accept、connect、read、write等，由processSelectedKeys方法触发。
- 非IO任务 添加到taskQueue中的任务，如register0、bind0等任务，由runAllTasks方法触发。

两种任务的执行时间比由变量ioRatio控制，默认为50，则表示允许非IO任务执行的时间与IO任务的执行时间相等。

NioEventLoopGroup

NioEventLoopGroup，主要管理eventLoop的生命周期，可以理解为一个线程池，内部维护了一组线程，每个线程(NioEventLoop)负责处理多个Channel上的事件，而一个Channel只对应于一个线程。

ChannelHandler

ChannelHandler是一个接口，处理I / O事件或拦截I / O操作，并将其转发到其ChannelPipeline(业务处理链)中的下一个处理程序。

ChannelHandler本身并没有提供很多方法，因为这个接口有许多的方法需要实现，方便使用期间，可以继承它的子类：

- ChannelInboundHandler用于处理入站I / O事件
- ChannelOutboundHandler用于处理出站I / O操作

或者使用以下适配器类：

- ChannelInboundHandlerAdapter用于处理入站I / O事件
- ChannelOutboundHandlerAdapter用于处理出站I / O操作
- ChannelDuplexHandler用于处理入站和出站事件

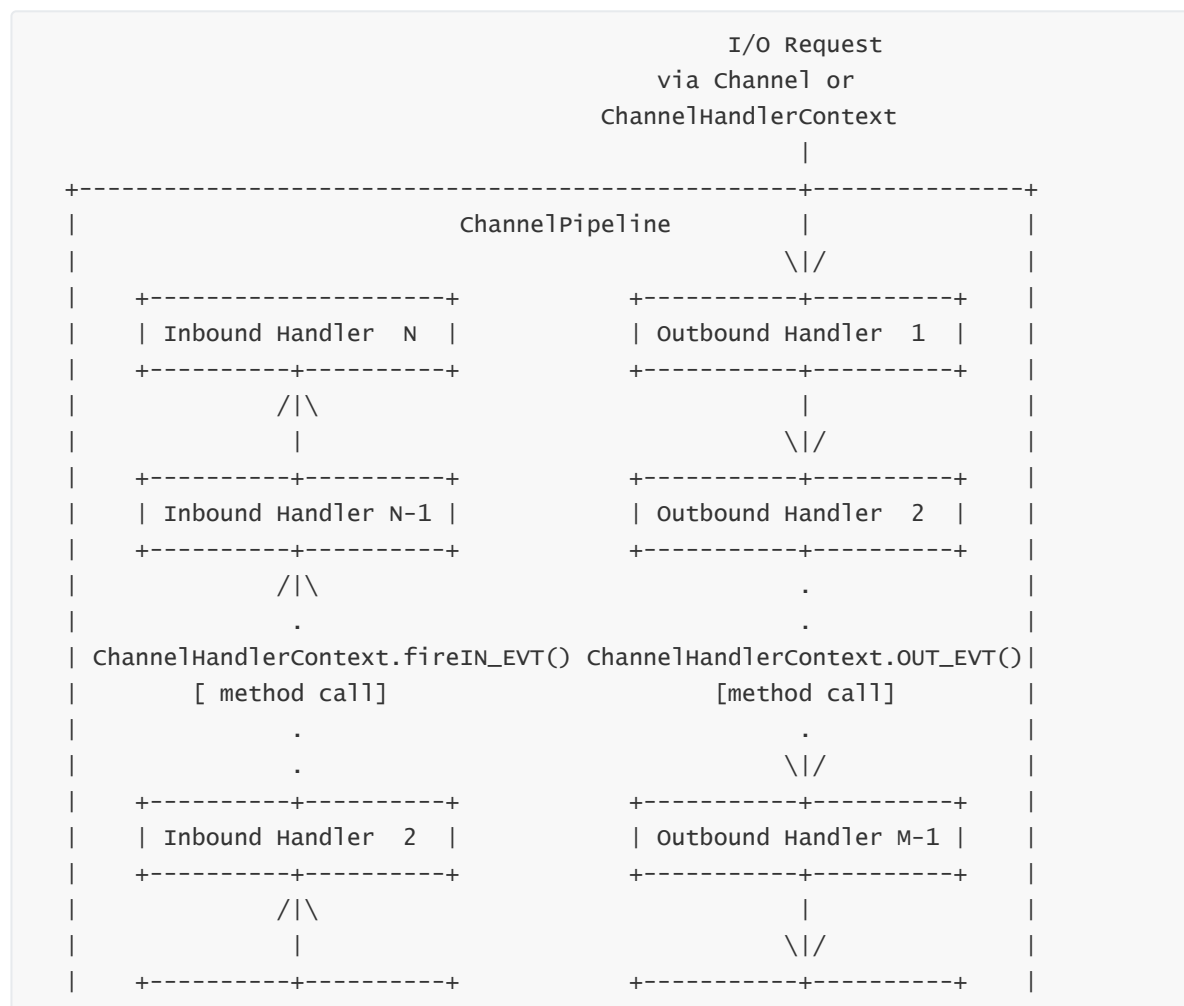
ChannelHandlerContext

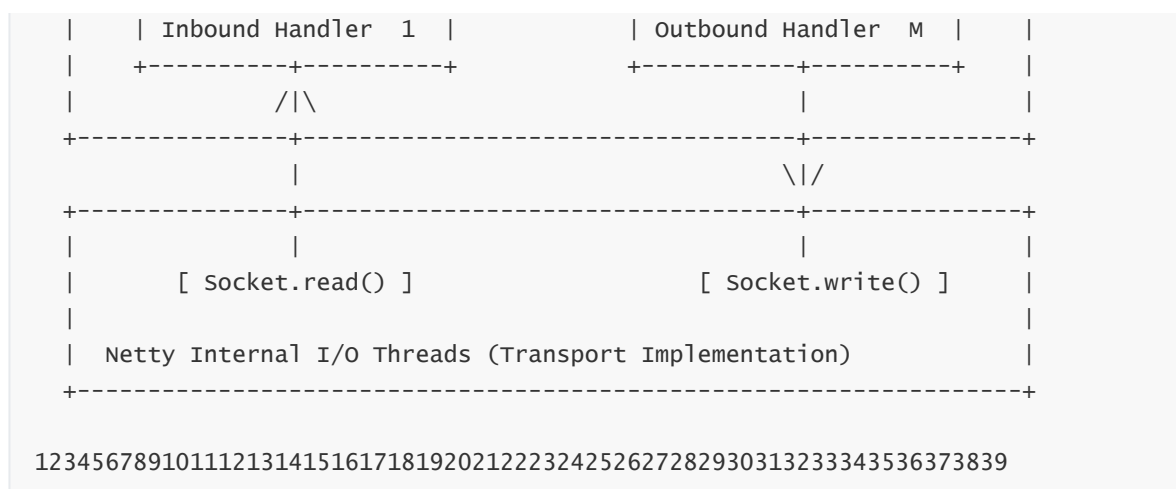
保存Channel相关的所有上下文信息，同时关联一个ChannelHandler对象

ChannelPipeline

保存ChannelHandler的List，用于处理或拦截Channel的入站事件和出站操作。ChannelPipeline实现了一种高级形式的拦截过滤器模式，使用户可以完全控制事件的处理方式，以及Channel中各个的ChannelHandler如何相互交互。

下图引用Netty的Javadoc4.1中ChannelPipeline的说明，描述了ChannelPipeline中ChannelHandler通常如何处理I/O事件。I/O事件由ChannelInboundHandler或ChannelOutboundHandler处理，并通过调用ChannelHandlerContext中定义的事件传播方法（例如ChannelHandlerContext.fireChannelRead（Object）和ChannelOutboundInvoker.write（Object））转发到其最近的处理程序。

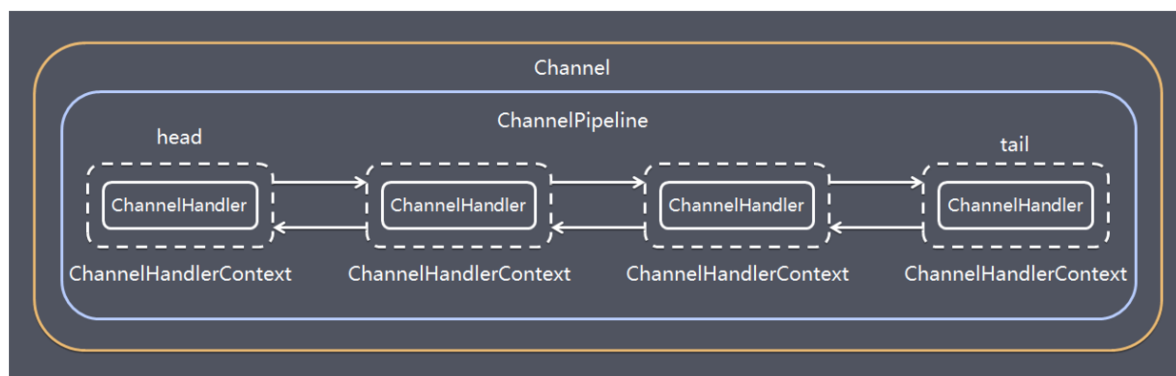




入站事件由自下而上方向的入站处理程序处理，如图左侧所示。入站Handler处理程序通常处理由图底部的I/O线程生成的入站数据。通常通过实际输入操作（例如SocketChannel.read（ByteBuffer））从远程读取入站数据。

出站事件由上下方向处理，如图右侧所示。出站Handler处理程序通常会生成或转换出站传输，例如write请求。I/O线程通常执行实际的输出操作，例如SocketChannel.write（ByteBuffer）。

在 Netty 中每个 Channel 都有且仅有一个 ChannelPipeline 与之对应, 它们的组成关系如下:



一个 Channel 包含了一个 ChannelPipeline, 而 ChannelPipeline 中又维护了一个由 ChannelHandlerContext 组成的双向链表, 并且每个 ChannelHandlerContext 中又关联着一个 ChannelHandler。入站事件和出站事件在一个双向链表中, 入站事件会从链表head往后传递到最后一个入站的handler, 出站事件会从链表tail往前传递到最前一个出站的handler, 两种类型的handler互不干扰。

聊聊：Netty服务端过程初始化并启动过程

初始化并启动Netty服务端过程如下:

```

public static void main(String[] args) {
    // 创建mainReactor
    NioEventLoopGroup bossGroup = new NioEventLoopGroup();
    // 创建工作线程组
    NioEventLoopGroup workerGroup = new NioEventLoopGroup();

    final ServerBootstrap serverBootstrap = new ServerBootstrap();
    serverBootstrap
        // 组装NioEventLoopGroup
        .group(bossGroup, workerGroup)
        // 设置channel类型为NIO类型

```

```

        .channel(NioServerSocketChannel.class)
        // 设置连接配置参数
        .option(ChannelOption.SO_BACKLOG, 1024)
        .childOption(ChannelOption.SO_KEEPALIVE, true)
        .childOption(ChannelOption.TCP_NODELAY, true)
        // 配置入站、出站事件handler
        .childHandler(new ChannelInitializer<NioSocketChannel>() {
            @Override
            protected void initChannel(NioSocketChannel ch) {
                // 配置入站、出站事件channel
                ch.pipeline().addLast(...);
                ch.pipeline().addLast(...);
            }
        });

    // 绑定端口
    int port = 8080;
    serverBootstrap.bind(port).addListener(future -> {
        if (future.isSuccess()) {
            System.out.println(new Date() + ": 端口[" + port + "]绑定成功!");
        } else {
            System.err.println("端口[" + port + "]绑定失败!");
        }
    });
}

```

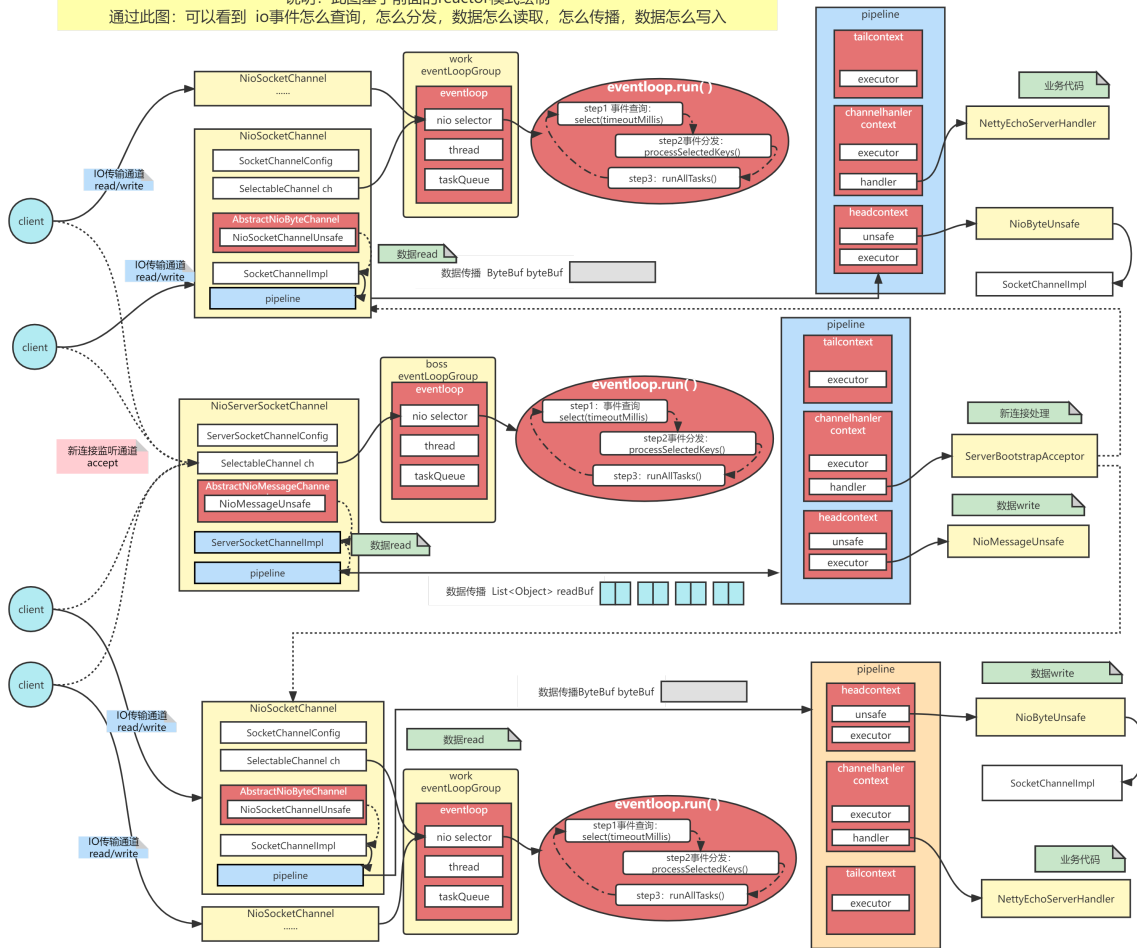
基本过程如下:

- 1 初始化创建2个NioEventLoopGroup, 其中boosGroup用于Accetpt连接建立事件并分发请求, workerGroup用于处理I/O读写事件和业务逻辑
- 2 基于ServerBootstrap(服务端启动引导类), 配置EventLoopGroup、Channel类型, 连接参数、配置入站、出站事件handler
- 3 绑定端口, 开始工作

服务端Netty的工作架构图

结合上面的介绍的Netty Reactor模型, 介绍服务端Netty的工作架构图:

说明：此图基于前面的reactor模式绘制
通过此图：可以看到 io事件怎么查询，怎么分发，数据怎么读取，怎么传播，数据怎么写入



server端包含1个Boss NioEventLoopGroup和1个Worker NioEventLoopGroup，

NioEventLoopGroup相当于1个事件循环组，这个组里包含多个事件循环NioEventLoop，每个NioEventLoop包含1个selector和1个事件循环线程。

每个Boss NioEventLoop循环执行的任务包含3步：

1 轮询accept事件

2 处理accept I/O事件，与Client建立连接，生成NioSocketChannel，并将NioSocketChannel注册到某个Worker NioEventLoop的Selector上 *3 处理任务队列中的任务，runAllTasks。

任务队列中的任务包括用户调用eventloop.execute或schedule执行的任务，或者其它线程提交到该eventloop的任务。

每个Worker NioEventLoop循环执行的任务包含3步：

1 轮询read、write事件；

2 处I/O事件，即read、write事件，在NioSocketChannel可读、可写事件发生时进行处理

3 处理任务队列中的任务，runAllTasks。

任务队列的使用场景

其中任务队列中的task有3种典型使用场景

1 用户程序自定义的普通任务

```
ctx.channel().eventLoop().execute(new Runnable() {  
    @Override  
    public void run() {  
        //...  
    }  
});
```

2 非当前reactor线程调用channel的各种方法

例如在推送系统的业务线程里面，根据用户的标识，找到对应的channel引用，然后调用write类方法向该用户推送消息，就会进入到这种场景。最终的write会提交到任务队列中后被异步消费。

3 用户自定义定时任务

```
ctx.channel().eventLoop().schedule(new Runnable() {  
    @Override  
    public void run() {  
        }  
    }, 60, TimeUnit.SECONDS);
```

聊一下：Netty ByteBuf的特点？

这里想要比较两种Buffer,对比ByteBuffer得出ByteBuf的优点点,

我们首先要做的就是总结ByteBuf的特点以及相比ByteBuffer,这个特点如何成为优点：

一：ByteBuf读写指针

在ByteBuffer中,读写指针都是position,

而在ByteBuf中,读写指针分别为readerIndex和writerIndex,

直观看上去ByteBuffer仅用了一个指针就实现了两个指针的功能,节省了变量,但是当对于ByteBuffer的读写状态切换的时候必须要调用flip方法,而当下一次写之前,必须要将Buffer中的内容读完,再调用clear方法。

每次读之前调用flip,写之前调用clear,这样无疑给开发带来了繁琐的步骤,而且内容没有读完是不能写的,这样非常不灵活。

相比之下我们看看ByteBuf,读的时候仅仅依赖readerIndex指针,写的时候仅仅依赖writerIndex指针,不需每次读写之前调用对应的方法,而且没有必须一次读完的限制。

二：ByteBuf引用计数

ByteBuf扩展了ReferenceCounted接口,这个接口定义的功能主要是引用计数：

ReferenceCounted接口定义

也就是所有对ByteBuf的实现,都要实现引用计数,Netty对Buffer资源进行了显式的管理,这部分要结合

Netty的内存池技术理解,当Buffer引用+1的时候,需要调用retain来让refCnt+1,当Buffer引用数-1的时候需要调用release来让refCnt-1,当refCnt变为0的时候Netty为pooled和unpooled的不同buffer提供了不同的实现,通常对于非内存池的用法,Netty把Buffer的内存回收交给了垃圾回收器,对于内存池的用法,Netty对内存的回收实际上是回收到内存池内,以提供下一次的申请所使用,关于内存池这部分可以参考我之前的一篇文章。

三：池化Buffer资源

由于Netty是一个NIO网络框架,因此对于Buffer的使用如果基于直接内存DirectBuffer: 实现的话,将会大大提高I/O操作的效率,然而DirectBuffer和HeapBuffer相比之下除了I/O操作效率高之外还有一个天生的缺点,即对于DirectBuffer的申请相比HeapBuffer效率更低,因此Netty结合引用计数实现了PolledBuffer,即池化的用法,当引用计数等于0的时候,Netty将Buffer回收致池中,在下一次申请Buffer的没某个时刻会被复用。Netty这样做的基本想法是我们花了很大的力气申请了一块内存,不能轻易让他被回收呀,能重复利用当然重复利用咯。

四：ByteBuffer才能和Channel打交道

归根结底,站在NIO的立场上所有的缓冲区要想和Channel打交道,换句话说也就是从网络Channel读取数据的时候,都是从Channel到ByteBuffer,从缓冲区写的网上上的时候,都是从ByteBuffer到Channel。因此,当Netty监听到I/O读事件的时候,会将自己流从Channel读到ByteBuffer而不是ByteBuf,see below:

```
return in.read((ByteBuffer) internalNioBuffer().clear().position(index).limit(index + length));
```

上面是ByteBuf的其中一个具体的读实现,可以看出ByteBuf维护着一个内部的ByteBuffer,叫做internalNioBuffer。当需要将字节流写入网络的时候,

需要将ByteBuf转换为ByteBuffer:

```
ByteBuffer tmpBuf;
if (internal) {
    tmpBuf = internalNioBuffer();
} else {
    tmpBuf = ByteBuffer.wrap(array);
}
return out.write((ByteBuffer) tmpBuf.clear().position(index).limit(index +
length));
}
```

简单聊聊：Netty的线程模型的三种使用方式？

单线程模型：

一个线程需要执行处理所有的 accept、read、decode、process、encode、send 事件。对于高负载、高并发，并且对性能要求比较高的场景不适用。

对应到 Netty 代码是下面这样的

使用 NioEventLoopGroup 类的无参构造函数设置线程数量的默认值就是 **CPU 核心数 2。

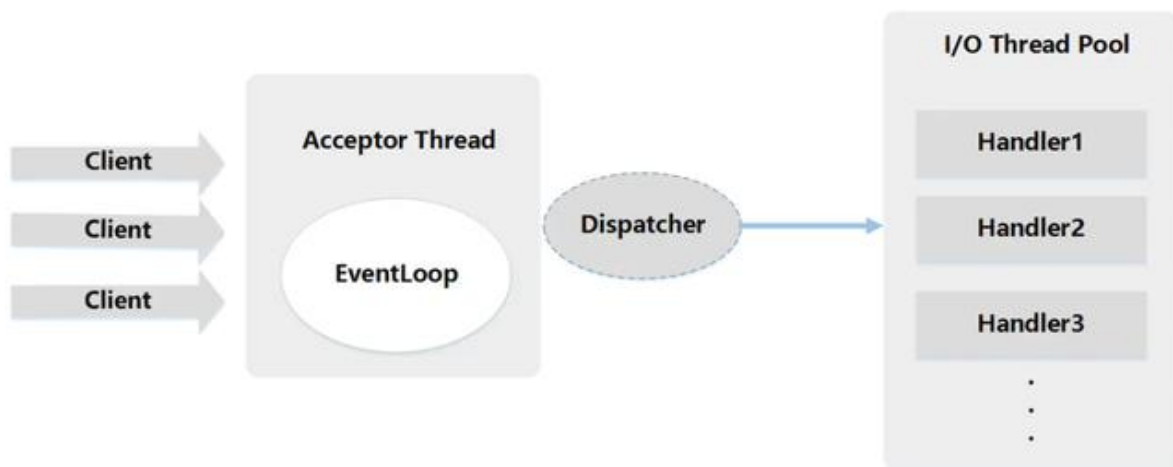
```
//1.eventGroup既用于处理客户端连接，又负责具体的处理。
EventLoopGroup eventGroup = new NioEventLoopGroup(1);
//2.创建服务端启动引导/辅助类：ServerBootstrap
ServerBootstrap b = new ServerBootstrap();
    boobtstrap.group(eventGroup, eventGroup)
//.....
```

多线程模型

一个 Acceptor 线程只负责监听客户端的连接，一个 NIO 线程池负责具体处理：accept、read、decode、process、encode、send 事件。满足绝大部分应用场景，并发连接量不大的时候没啥问题，但是遇到并发连接大的时候就可能会出现性能问题，成为性能瓶颈。

对应到 Netty 代码是下面这样的：

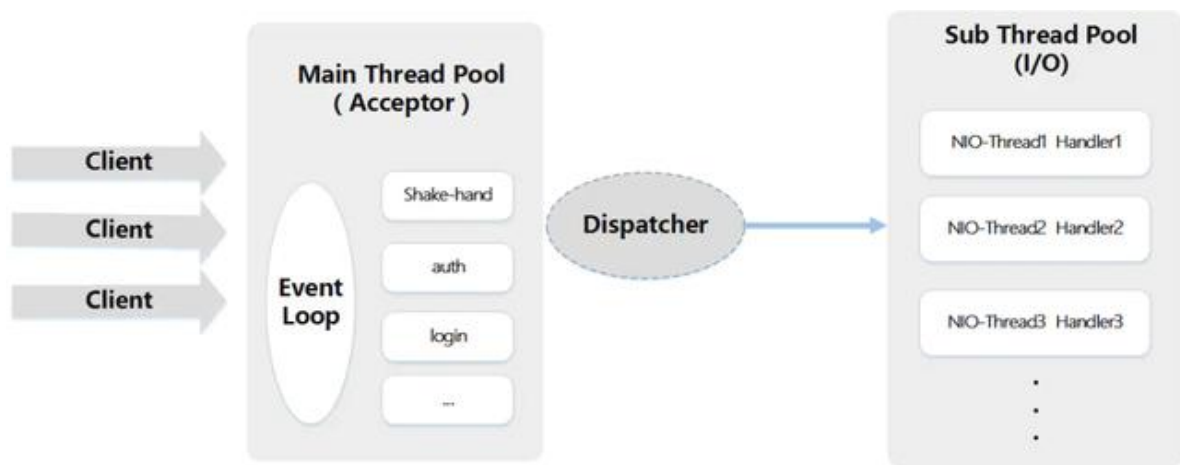
```
// 1.bossGroup 用于接收连接，workerGroup 用于具体的处理
EventLoopGroup bossGroup = new NioEventLoopGroup(1);
EventLoopGroup workerGroup = new NioEventLoopGroup();
try {
    //2.创建服务端启动引导/辅助类：ServerBootstrap
    ServerBootstrap b = new ServerBootstrap();
    //3.给引导类配置两大线程组，确定了线程模型
    b.group(bossGroup, workerGroup)
    //.....
}
```



主从多线程模型

从一个主线程 NIO 线程池中选择一个线程作为 Acceptor 线程，绑定监听端口，接收客户端连接，其他线程负责后续的接入认证等工作。连接建立完成后，Sub NIO 线程池负责具体处理 I/O 读写。如果多线程模型无法满足你的需求的时候，可以考虑使用主从多线程模型。

```
// 1.bossGroup 用于接收连接，workerGroup 用于具体的处理
EventLoopGroup bossGroup = new NioEventLoopGroup();
EventLoopGroup workerGroup = new NioEventLoopGroup();
try {
    //2.创建服务端启动引导/辅助类：ServerBootstrap
    ServerBootstrap b = new ServerBootstrap();
    //3.给引导类配置两大线程组，确定了线程模型
    b.group(bossGroup, workerGroup)
    //.....
}
```



默认情况 Netty 起多少线程？何时启动？

Netty 默认是 CPU 处理器数的两倍，bind 完之后启动。

TCP 粘包/拆包的原因及解决方法？

TCP粘包/分包的原因：

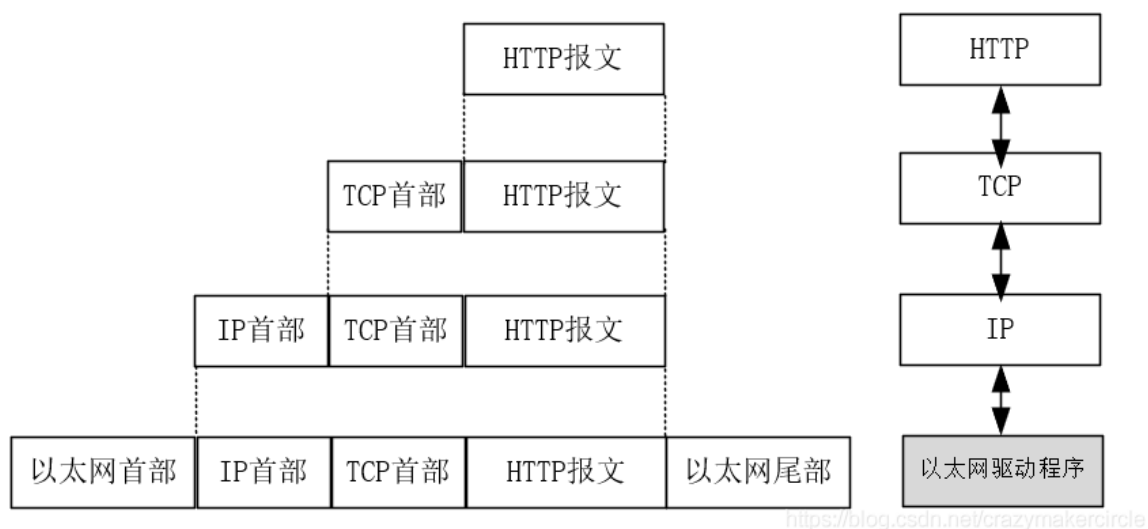
TCP是以流的方式来处理数据，一个完整的包可能会被TCP拆分成多个包进行发送，也可能把小的封装成一个大的数据包发送。

深层次的原因，是TCP/IP协议，是四层协议，会逐层进行数据的封装和分用。

而每一层都有自己的特定标识和头部，每一层的数据

数据通过互联网传输的时候不可能是光秃秃的不加标识，如果这样数据就会乱。所以数据在发送的时候，需要加上特定标识，加上特定标识的过程叫做数据的封装，在数据使用的时候再去掉特定标识，去掉特定标识的过程就叫做分用。

TCP/IP协议的数据封装和分用过程，大致如下图所示：



图：TCP/IP协议的数据封装和分用过程TCP/IP协议是层层

参考[TCP/IP协议（图解+秒懂+史上最全）](#)

在TCP/IP分层中，OSI的应用层、表示层、会话层全都统一为应用层，而底层的物理层和链路层又统一为接口层。每个TCP/IP分层都对应一些协议，这些协议组成了TCP/IP协议栈。

最顶层的是应用层协议，比如HTTP协议、DNS协议、FTP协议、POP3协议等。

传输层包含两个协议，TCP协议和UDP协议，在这一层将指定通信端口。

网络层包括ARP协议、IP协议、ICMP协议、IGMP协议等等，网络层的协议有些并不完全属于网络层，有可能向下跨到链路层，但总的来说，将它们都归类到网络层协议。

以太网：每个以太网帧最小64字节，最大帧长为1500字节，以太网帧的格式如下图所示。

注意，上图中只有链路层部分才是以太网帧的数据，所以7字节的前同步码（Preamble）、1字节的帧起始定界符、12字节的帧间距不属于以太网帧，它们属于物理层。其中，帧间距的存在，说明每个帧之间是间隔传输的，并不是完全连续传输。

主要看以太网帧格式。首先是6字节的目标MAC地址和6字节的源MAC地址，然后是可选的tag标记，一般当它不存在，之后是表示类型的2个字节，接着便是数据包部分，尾部是一个4字节CRC的帧校验码。

目标MAC字段记录了目标的MAC地址，当某网卡或适配器接收到了一个以太网帧，如果它的MAC地址正好是帧中记录的目标地址，则将该帧解封后的数据包传递给网络层。此外，如果收到的是广播帧（目标MAC地址为FF-FF-FF-FF-FF-FF），则也将其传递给网络层，但如果收到目标MAC地址不是本接口地址的任何数据帧，都将直接丢弃。

源MAC地址字段记录的是该帧的出口MAC地址。

由于信道是所有主机共享的，如果数据帧太长就会出现有的主机长时间不能发送数据，而且有的发送数据可能超出接收端的缓冲区大小，造成缓冲溢出。为避免单一主机占用信道时间过长，规定了以太网帧的最大帧长为1500。

由于信道是所有主机共享的，如果数据帧太长就会出现有的主机长时间不能发送数据，而且有的发送数据可能超出接收端的缓冲区大小，造成缓冲溢出。为避免单一主机占用信道时间过长，规定了以太网帧的最大帧长为1500。

MTU

在网络中，最大传输单元（MTU）指通过联网设备可以接收的最大数据包的值。

MTU可以想象为高速公路地下通道或隧道的高度限制：超过高度限制的汽车和卡车无法通过，就像超过网络 MTU 的数据包无法通过该网络一样。

不过，与汽车和卡车不同的是，超过 MTU 的数据包可被分解成较小的碎片，从而能通过网络。

这个过程称为分片。分片的数据包在到达目的地后会重新组装。

MTU 以字节数为单位，一个“字节”等于 8 位信息，即 8 个一和零。1,500 字节是最大 MTU 大小。

MSS

MSS: Maximum Segment Size 最大报文段长度

MSS是最大报文段长度的缩写，是TCP协议里面的一个概念。

MSS是TCP[数据包]每次能够传输的最大数据分段。

为了达到最佳的传输效能，TCP协议在建立连接的时候通常要协商双方的MSS值，

这个值TCP协议在实现的时候往往用MTU值代替（需要减去IP数据包包头的大小20Bytes和TCP数据段的包头20Bytes）所以一般MSS值1460

通讯双方会根据双方提供的MSS值得最小值确定为这次连接的最大MSS值。

注：最大报文段长度MSS这个名词很容易引起误解。

MSS是TCP报文段中的数据字段的最大长度。数据字段加上TCP首部才等于整个的TCP报文段。所以MSS并不是TCP报文段的最大长度，而是：

$MSS = \text{TCP报文段长度} - \text{TCP首部长度} - \text{IP首部}$

MSS（Maximum Segment Size，最大报文长度），是TCP协议定义的一个选项，MSS选项用于在TCP连接建立时，收发双方协商通信时每一个报文段所能承载的最大数据长度。

应用层的半包问题（拆包和粘包）

拆包：一个完整的应用包可能会被 TCP 拆分成多个包进行发送。

粘包：也可能把小应用包的封装成一个大的 TCP数据包发送。

应用程序写入的字节大小大于套接字发送缓冲区的大小，会发生拆包现象，

而应用程序写入数据小于套接字缓冲区大小，网卡将应用多次写入的数据发送到网络上，这将会发生粘包现象。

实际上，拆包、粘包的本质，就是受限于各层协议净荷的大小不一，

都要对上层的数据进行重新组装，

上层的协议收到下层的数据包，都有可能出现拆包、粘包问题。

当TCP帧的payload（净荷）TCP报文长度-TCP头部长度>MSS (1460)的时候， 将发生拆包，进行MSS大小的TCP分段，

以太网帧的payload（净荷）大于MTU（1500字节）进行以太网帧 分片。

拆包和粘包解决方法

解决方法

Netty中提供了多个 Decoder 解析类 用于解决上述问题：

- FixedLengthFrameDecoder、LengthFieldBasedFrameDecoder，固定长度是消息头指定消息长度的一种形式，进行粘包拆包处理的。
- LineBasedFrameDecoder、DelimiterBasedFrameDecoder，换行是于指定消息边界方式的一种形式，进行消息粘

Netty 中有哪种重要组件？

- Channel：Netty 网络操作抽象类，它除了包括基本的 I/O 操作，如 bind、connect、read、write 等。
- EventLoop：主要是配合 Channel 处理 I/O 操作，用来处理连接的生命周期中所发生的事情。
- ChannelFuture：Netty 框架中所有的 I/O 操作都为异步的，因此我们需要 ChannelFuture 的 addListener()注册一个 ChannelFutureListener 监听事件，当操作执行成功或者失败时，监听就会自动触发返回结果。
- ChannelHandler：充当了所有处理入站和出站数据的逻辑容器。ChannelHandler 主要用来处理各种事件，这里的事件很广泛，比如可以是连接、数据接收、异常、数据转换等。
- ChannelPipeline：为 ChannelHandler 链提供了容器，当 channel 创建时，就会被自动分配到它专属的 ChannelPipeline，这个关联是永久性的。

Netty 发送消息有几种方式？

Netty 有两种发送消息的方式：

- 直接写入 Channel 中，消息从 ChannelPipeline 当中尾部开始移动；
- 写入和 ChannelHandler 绑定的 ChannelHandlerContext 中，消息从 ChannelPipeline 中的下一个 ChannelHandler 中移动。

什么是 Netty 的零拷贝？

Netty 的零拷贝主要包含三个方面：

- Netty 的接收和发送 ByteBuffer 采用 DIRECT BUFFERS，使用堆外直接内存进行 Socket 读写，不需要进行字节缓冲区的二次拷贝。如果使用传统的堆内存（HEAP BUFFERS）进行 Socket 读写，

JVM 会将堆内存 Buffer 拷贝一份到直接内存中，然后才写入 Socket 中。相比于堆外直接内存，消息在发送过程中多了一次缓冲区的内存拷贝。

- Netty 提供了组合 Buffer 对象，可以聚合多个 ByteBuffer 对象，用户可以像操作一个 Buffer 那样方便的对组合 Buffer 进行操作，避免了传统通过内存拷贝的方式将几个小 Buffer 合并成一个大的 Buffer。
- Netty 的文件传输采用了 transferTo 方法，它可以直接将文件缓冲区的数据发送到目标 Channel，避免了传统通过循环 write 方式导致的内存拷贝问题。

这个答案不完整，具体请参考 视频

了解哪几种序列化协议？

序列化（编码）是将对象序列化为二进制形式（字节数组），主要用于网络传输、数据持久化等；而反序列化（解码）则是将从网络、磁盘等读取的字节数组还原成原始对象，主要用于网络传输对象的解码，以便完成远程调用。

影响序列化性能的关键因素：序列化后的码流大小（网络带宽的占用）、序列化的性能（CPU资源占用）；是否支持跨语言（异构系统的对接和开发语言切换）。

Java默认提供的序列化：无法跨语言、序列化后的码流太大、序列化的性能差

XML，优点：人机可读性好，可指定元素或特性的名称。缺点：序列化数据只包含数据本身以及类的结构，不包括类型标识和程序集信息；只能序列化公共属性和字段；不能序列化方法；文件庞大，文件格式复杂，传输占带宽。适用场景：当做配置文件存储数据，实时数据转换。

JSON，是一种轻量级的数据交换格式，优点：兼容性高、数据格式比较简单，易于读写、序列化后数据较小，可扩展性好，兼容性好、与XML相比，其协议比较简单，解析速度比较快。缺点：数据的描述性比XML差、不适合性能要求为ms级别的情况、额外空间开销比较大。适用场景（可替代XML）：跨防火墙访问、可调式性要求高、基于Web browser的Ajax请求、传输数据量相对小，实时性要求相对低（例如秒级别）的服务。

Fastjson，采用一种“假定有序快速匹配”的算法。优点：接口简单易用、目前java语言中最快的json库。缺点：过于注重快，而偏离了“标准”及功能性、代码质量不高，文档不全。适用场景：协议交互、Web输出、Android客户端

Thrift，不仅是序列化协议，还是一个RPC框架。优点：序列化后的体积小，速度快、支持多种语言和丰富的数据类型、对于数据字段的增删具有较强的兼容性、支持二进制压缩编码。缺点：使用者较少、跨防火墙访问时，不安全、不具有可读性，调试代码时相对困难、不能与其他传输层协议共同使用（例如HTTP）、无法支持向持久层直接读写数据，即不适合做数据持久化序列化协议。适用场景：分布式系统的RPC解决方案

Avro，Hadoop的一个子项目，解决了JSON的冗长和没有IDL的问题。优点：支持丰富的数据类型、简单的动态语言结合功能、具有自我描述属性、提高了数据解析速度、快速可压缩的二进制数据形式、可以实现远程过程调用RPC、支持跨编程语言实现。缺点：对于习惯于静态类型语言的用户不直观。适用场景：在Hadoop中做Hive、Pig和MapReduce的持久化数据格式。

Protobuf，将数据结构以.proto文件进行描述，通过代码生成工具可以生成对应数据结构的POJO对象和Protobuf相关的方法和属性。优点：序列化后码流小，性能高、结构化数据存储格式（XML JSON等）、通过标识字段的顺序，可以实现协议的前向兼容、结构化的文档更容易管理和维护。缺点：需要依赖于工具生成代码、支持的语言相对较少，官方只支持Java、C++、python。适用场景：对性能要

求高的RPC调用、具有良好的跨防火墙的访问属性、适合应用层对象的持久化

其它

protostuff 基于protobuf协议，但不需要配置proto文件，直接导包即可

Jboss marshaling 可以直接序列化java类，无须实现java.io.Serializable接口

Message pack 一个高效的二进制序列化格式

Hessian 采用二进制协议的轻量级remoting onhttp工具

kryo 基于protobuf协议，只支持java语言,需要注册（Registration），然后序列化（Output），反序列化（Input）

如何选择序列化协议？

具体场景

对于公司间的系统调用，如果性能要求在100ms以上的服务，基于XML的SOAP协议是一个值得考虑的方案。

基于Web browser的Ajax，以及Mobile app与服务端之间的通讯，JSON协议是首选。对于性能要求不太高，或者以动态类型语言为主，或者传输数据载荷很小的运用场景，JSON也是非常不错的选择。

对于调试环境比较恶劣的场景，采用JSON或XML能够极大的提高调试效率，降低系统开发成本。

当对性能和简洁性有极高要求的场景，Protobuf, Thrift, Avro之间具有一定的竞争关系。

对于T级别的数据的持久化应用场景，Protobuf和Avro是首要选择。如果持久化后的数据存储在hadoop子项目里，Avro会是更好的选择。

对于持久层非Hadoop项目，以静态类型语言为主的应用场景，Protobuf会更符合静态类型语言工程师的开发习惯。由于Avro的设计理念偏向于动态类型语言，对于动态语言为主的应用场景，Avro是更好的选择。

如果需要提供一个完整的RPC解决方案，Thrift是一个好的选择。

如果序列化之后需要支持不同的传输层协议，或者需要跨防火墙访问的高性能场景，Protobuf可以优先考虑。

protobuf的数据类型有多种：bool、double、float、int32、int64、string、bytes、enum、message。protobuf的限定符：required: 必须赋值，不能为空、optional: 字段可以赋值，也可以不赋值、repeated: 该字段可以重复任意次数（包括0次）、枚举；只能用指定的常量集中的一个值作为其值；

protobuf的基本规则：每个消息中必须至少留有一个required类型的字段、包含0个或多个optional类型的字段；repeated表示的字段可以包含0个或多个数据；[1,15]之内的标识号在编码的时候会占用一个字节（常用），[16,2047]之内的标识号则占用2个字节，标识号一定不能重复、使用消息类型，也可以将消息嵌套任意多层，可用嵌套消息类型来代替组。

protobuf的消息升级原则：不要更改任何已有的字段的数值标识；不能移除已经存在的required字段，optional和repeated类型的字段可以被移除，但要保留标号不能被重用。新添加的字段必须是optional或repeated。因为旧版本程序无法读取或写入新增的required限定符的字段。

编译器为每一个消息类型生成了一个.java文件，以及一个特殊的Builder类（该类是用来创建消息类接口的）。如：UserProto.User.Builder builder = UserProto.User.newBuilder();builder.build();

Netty中的使用：ProtobufVarint32FrameDecoder 是用于处理半包消息的解码类；

ProtobufDecoder(UserProto.User.getDefaultInstance())这是创建的用户Proto.java文件中的解码

类；ProtobufVarint32LengthFieldPrepender 对protobuf协议的消息头上加上一个长度为32的整形字段，用于标志这个消息的长度的类；ProtobufEncoder 是编码类

将StringBuilder转换为ByteBuf类型：copiedBuffer()方法

Netty 支持哪些心跳类型设置？

- readerIdleTime：为读超时时间（即测试端一定时间内未接受到被测试端消息）。
- writerIdleTime：为写超时时间（即测试端一定时间内向被测试端发送消息）。
- allIdleTime：所有类型的超时时间。

为什么没有使用Netty5？

现在稳定推荐使用的主流版本还是Netty4，

Netty5 中使用了 ForkJoinPool，增加了代码的复杂度，但是对性能的改善却不明显，

所以Netty5 版本不推荐使用，官网也没有提供下载链接。

Netty 入门门槛相对较高，其实是因为这方面的资料较少，并不是因为他有多难，大家其实都可以像搞透 Spring 一样搞透 Netty。

在学习之前，建议先理解透整个框架原理结构，运行过程，可以少走很多弯路。

简单说说：NIOEventLoopGroup源码？

NioEventLoopGroup(其实是MultithreadEventExecutorGroup) 内部维护一个类型为 EventExecutor children []，

默认大小是处理器核数 * 2，这样就构成了一个线程池，初始化EventExecutor时NioEventLoopGroup重载newChild方法，所以children元素的实际类型为NioEventLoop。

线程启动时调用SingleThreadEventExecutor的构造方法，执行NioEventLoop类的run方法，首先会调用hasTasks()方法判断当前taskQueue是否有元素。

如果taskQueue中有元素，执行 selectNow() 方法，最终执行selector.selectNow()，该方法会立即返回。如果taskQueue没有元素，执行 select(oldWakenUp) 方法

select (oldWakenUp) 方法解决了 Nio 中的 bug，具体的方案为：

selectCnt 用来记录selector.select方法的执行次数和标识是否执行过selector.selectNow()，若触发了epoll的空轮询bug，则会反复执行selector.select(timeoutMillis)，变量selectCnt 会逐渐变大，当selectCnt 达到阈值（默认512），则执行rebuildSelector方法，进行selector重建，解决cpu占用100%的bug。

rebuildSelector方法先通过openSelector方法创建一个新的selector。然后将old selector的selectionKey执行cancel。最后将old selector的channel重新注册到新的selector中。rebuild后，需要重新执行方法selectNow，检查是否有已ready的selectionKey。

接下来调用processSelectedKeys 方法（处理I/O任务），当selectedKeys != null时，调用processSelectedKeysOptimized方法，迭代 selectedKeys 获取就绪的 IO 事件的selectkey存放在数组selectedKeys中, 然后为每个事件都调用 processSelectedKey 来处理它，processSelectedKey 中分别处理OP_READ；OP_WRITE；OP_CONNECT事件。

最后调用runAllTasks方法（非IO任务），该方法首先会调用fetchFromScheduledTaskQueue方法，把scheduledTaskQueue中已经超过延迟执行时间的任务移到taskQueue中等待被执行，然后依次从taskQueue中取任务执行，每执行64个任务，进行耗时检查，如果已执行时间超过预先设定的执行时间，则停止执行非IO任务，避免非IO任务太多，影响IO任务的执行。

每个NioEventLoop对应一个线程和一个Selector，NioServerSocketChannel会主动注册到某一个NioEventLoop的Selector上，NioEventLoop负责事件轮询。

Outbound 事件都是请求事件, 发起者是 Channel, 处理者是 unsafe, 通过 Outbound 事件进行通知, 传播方向是 tail到head。Inbound 事件发起者是 unsafe, 事件的处理者是 Channel, 是通知事件, 传播方向是从头到尾。

内存管理机制，首先会预申请一大块内存Arena，Arena由许多Chunk组成，而每个Chunk默认由2048个page组成。Chunk通过AVL树的形式组织Page，每个叶子节点表示一个Page，而中间节点表示内存区域，节点自己记录它在整个Arena中的偏移地址。当区域被分配出去后，中间节点上的标记位会被标记，这样就表示这个中间节点以下的所有节点都已被分配了。大于8k的内存分配在poolChunkList中，而PoolSubpage用于分配小于8k的内存，它会把一个page分割成多段，进行内存分配。

ByteBuf的特点：支持自动扩容（4M），保证put方法不会抛出异常、通过内置的复合缓冲类型，实现零拷贝（zero-copy）；不需要调用flip()来切换读/写模式，读取和写入索引分开；方法链；引用计数基于AtomicIntegerFieldUpdater用于内存回收；PooledByteBuf采用二叉树来实现一个内存池，集中管理内存的分配和释放，不用每次使用都新建一个缓冲区对象。UnpooledHeapByteBuf每次都会新建一个缓冲区对象。

问题：JDK NIO中Epoll空轮询Bug产生的原因以及Netty中是如何解决的

产生原因

```

ServerSocketChannel serverChannel = ServerSocketChannel.open();
serverChannel.configureBlocking(false);
ServerSocket server = serverChannel.socket();
server.bind(new InetSocketAddress(port));

Selector selector = Selector.open();
serverChannel.register(selector, SelectionKey.OP_ACCEPT);
SocketChannel clientChannel = null;

System.out.println("0. SERVER STARTED TO LISTEN");
boolean writeNow = false;

while (true) {
    try {
        // wait for selection
        int numKeys = selector.select(); 通常是阻塞的 ==》但是这个时候
        if (numKeys == 0) { select却被唤醒
            System.err.println("select wakes up with zero!!!");
        }

        Iterator it = selector.selectedKeys().iterator();
        while (it.hasNext()) { 遍历出事件的key, 然后循环进行操作
            SelectionKey selected = (SelectionKey) it.next();
            int ops = selected interestOps();

            try {
                // process new connection
                if ((ops & SelectionKey.OP_ACCEPT) != 0) {
                    clientChannel = serverChannel.accept();
                    clientChannel.configureBlocking(false);

                    // register channel to selector
                    clientChannel.register(selector, SelectionKey.OP_READ, null);
                    System.out.println("2. SERVER ACCEPTED AND REGISTER READ OP : " +
                        clientChannel.socket().getInetAddress());
                }

                if ((ops & SelectionKey.OP_READ) != 0) {
                    // read client message
                    System.out.println("3. SERVER READ DATA FROM client - " +
                        clientChannel.socket().getInetAddress());
                    readClient((SocketChannel) selected.channel(), buffer);

                    // deregister OP_READ
                    System.out.println("PREV SET : " + selected.interestOps());
                    selected.interestOps(selected.interestOps() &
                        ~SelectionKey.OP_READ);
                    System.out.println("NEW SET : " + selected.interestOps());

                    Thread.sleep(SLEEP_PERIOD * 2);
                    new WriterThread(clientChannel).start();
                }
            } catch (IOException e) {
                // ...
            }
        }
    } catch (InterruptedException e) {
        // ...
    }
}

```

从而导致不断执行这个while

因为key==0, 所以这段循环根本不会执行

正常情况下, selector.select()操作是阻塞的, 只有被监听的fd有读写操作时, 才被唤醒。

但是, 在这个bug中, 没有任何fd有读写请求, 但是select()操作依旧被唤醒很显然, 这种情况下, selectedKeys()返回的是个空数组, 然后按照逻辑执行到while(true)处, 循环执行, 导致死循环。

Netty的解决方法

1. 对Selector的select操作周期进行统计, 每完成一次空的select操作进行一次计数。
2. 若在某个周期内连续发生N次空轮询, 则触发了epoll死循环bug。
3. 重建Selector, 判断是否是其他线程发起的重建请求, 若不是则将原SocketChannel从旧的Selector上去除注册, 重新注册到新的Selector上, 并将原来的Selector关闭。

尼恩暗语:

以上的问题和答案, 来自于网络(一搜一大把), 并且点击率非常高。

以上问题, 出现了知识性错误, 不是答案有误, 而是问题有误。

因为：是select空轮训导致cpu100% 的问题，二epoll并没有出现空轮训导致cpu100% 的问题
所以，使用epoll 反应器就可以了

简单说说：Netty如何解决Selector空轮询BUG的策略

这是早期，小伙伴在卷王群里，提到的高频面试题，直接贴链接哈

[Netty解决Selector空轮询BUG的策略（图解+秒懂+史上最全）](#)

Netty如何实现断线重连？

在实现TCP长连接功能中，客户端断线重连是一个很常见的问题，当我们使用netty实现断线重连时，是否考虑过如下几个问题：

- 如何监听到客户端和服务端连接断开？
- 如何实现断线后重新连接？
- netty客户端线程给多大比较合理？

实现思路

客户端在监测到与服务器端的连接断开后，或者一开始就无法连接的情况下，使用指定的重连策略进行重连操作，直到重新建立连接或重试次数耗尽。

对于如何监测连接是否断开，则是通过重写ChannelInboundHandler#channelInactive来实现，但连接不可用，该方法会被触发，所以只需要在该方法做好重连工作即可。

代码实现

注：以下代码都是在上面心跳机制的基础上修改/添加的。

因为断线重连是客户端的工作，所以只需对客户端代码进行修改。

重试策略

RetryPolicy —— 重试策略接口

```
public interface RetryPolicy {

    /**
     * Called when an operation has failed for some reason. This method should
     return
     * true to make another attempt.
     *
     * @param retryCount the number of times retried so far (0 the first time)
     * @return true/false
     */
    boolean allowRetry(int retryCount);

    /**
     * get sleep time in ms of current retry count.
     */
}
```

```

    *
    * @param retryCount current retry count
    * @return the time to sleep
    */
    long getSleepTimeMs(int retryCount);
}

```

ExponentialBackOffRetry —— 重连策略的默认实现

```

/**
 * <p>Retry policy that retries a set number of times with increasing sleep time
 * between retries</p>
 */
public class ExponentialBackOffRetry implements RetryPolicy {

    private static final int MAX_RETRIES_LIMIT = 29;
    private static final int DEFAULT_MAX_SLEEP_MS = Integer.MAX_VALUE;

    private final Random random = new Random();
    private final long baseSleepTimeMs;
    private final int maxRetries;
    private final int maxSleepMs;

    public ExponentialBackOffRetry(int baseSleepTimeMs, int maxRetries) {
        this(baseSleepTimeMs, maxRetries, DEFAULT_MAX_SLEEP_MS);
    }

    public ExponentialBackOffRetry(int baseSleepTimeMs, int maxRetries, int
maxSleepMs) {
        this.maxRetries = maxRetries;
        this.baseSleepTimeMs = baseSleepTimeMs;
        this.maxSleepMs = maxSleepMs;
    }

    @Override
    public boolean allowRetry(int retryCount) {
        if (retryCount < maxRetries) {
            return true;
        }
        return false;
    }

    @Override
    public long getSleepTimeMs(int retryCount) {
        if (retryCount < 0) {
            throw new IllegalArgumentException("retries count must greater than
0.");
        }
        if (retryCount > MAX_RETRIES_LIMIT) {
            System.out.println(String.format("maxRetries too large (%d). Pinning
to %d", maxRetries, MAX_RETRIES_LIMIT));
            retryCount = MAX_RETRIES_LIMIT;
        }
        long sleepMs = baseSleepTimeMs * Math.max(1, random.nextInt(1 <<
retryCount));
        if (sleepMs > maxSleepMs) {

```

```

        System.out.println(String.format("Sleep extension too large (%d).
Pinning to %d", sleepMs, maxSleepMs));
        sleepMs = maxSleepMs;
    }
    return sleepMs;
}
}

```

ReconnectHandler——重连处理器

```

@ChannelHandler.Sharable
public class ReconnectHandler extends ChannelInboundHandlerAdapter {

    private int retries = 0;
    private RetryPolicy retryPolicy;

    private TcpClient tcpClient;

    public ReconnectHandler(TcpClient tcpClient) {
        this.tcpClient = tcpClient;
    }

    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        System.out.println("Successfully established a connection to the
server.");
        retries = 0;
        ctx.fireChannelActive();
    }

    @Override
    public void channelInactive(ChannelHandlerContext ctx) throws Exception {
        if (retries == 0) {
            System.err.println("Lost the TCP connection with the server.");
            ctx.close();
        }

        boolean allowRetry = getRetryPolicy().allowRetry(retries);
        if (allowRetry) {

            long sleepTimeMs = getRetryPolicy().getSleepTimeMs(retries);

            System.out.println(String.format("Try to reconnect to the server
after %dms. Retry count: %d.", sleepTimeMs, ++retries));

            final EventLoop eventLoop = ctx.channel().eventLoop();
            eventLoop.schedule(() -> {
                System.out.println("Reconnecting ...");
                tcpClient.connect();
            }, sleepTimeMs, TimeUnit.MILLISECONDS);
        }
        ctx.fireChannelInactive();
    }

    private RetryPolicy getRetryPolicy() {
        if (this.retryPolicy == null) {
            this.retryPolicy = tcpClient.getRetryPolicy();
        }
    }
}

```

```

    }
    return this.retryPolicy;
}
}
}

```

ClientHandlersInitializer

在之前的基础上，添加了重连处理器ReconnectHandler。

```

public class ClientHandlersInitializer extends ChannelInitializer<SocketChannel>
{
    private ReconnectHandler reconnectHandler;
    private EchoHandler echoHandler;

    public ClientHandlersInitializer(TcpClient tcpClient) {
        Assert.notNull(tcpClient, "TcpClient can not be null.");
        this.reconnectHandler = new ReconnectHandler(tcpClient);
        this.echoHandler = new EchoHandler();
    }

    @Override
    protected void initChannel(SocketChannel ch) throws Exception {
        ChannelPipeline pipeline = ch.pipeline();
        pipeline.addLast(this.reconnectHandler);
        pipeline.addLast(new LengthFieldBasedFrameDecoder(Integer.MAX_VALUE, 0,
4, 0, 4));
        pipeline.addLast(new LengthFieldPrepender(4));
        pipeline.addLast(new StringDecoder(CharsetUtil.UTF_8));
        pipeline.addLast(new StringEncoder(CharsetUtil.UTF_8));
        pipeline.addLast(new Pinger());
    }
}

```

TcpClient

在之前的基础上添加重连、重连策略的支持。

```

public class TcpClient {

    private String host;
    private int port;
    private Bootstrap bootstrap;
    /** 重连策略 */
    private RetryPolicy retryPolicy;
    /** 将<code>Channel</code>保存起来，可用于在其他非handler的地方发送数据 */
    private Channel channel;

    public TcpClient(String host, int port) {
        this(host, port, new ExponentialBackOffRetry(1000, Integer.MAX_VALUE, 60
* 1000));
    }

    public TcpClient(String host, int port, RetryPolicy retryPolicy) {
        this.host = host;
        this.port = port;
        this.retryPolicy = retryPolicy;
    }
}

```

```

        init();
    }

    /**
     * 向远程TCP服务器请求连接
     */
    public void connect() {
        synchronized (bootstrap) {
            ChannelFuture future = bootstrap.connect(host, port);
            future.addListener(getConnectionListener());
            this.channel = future.channel();
        }
    }

    public RetryPolicy getRetryPolicy() {
        return retryPolicy;
    }

    private void init() {
        EventLoopGroup group = new NioEventLoopGroup();
        // bootstrap 可重用，只需在TcpClient实例化的时候初始化即可。
        bootstrap = new Bootstrap();
        bootstrap.group(group)
            .channel(NioSocketChannel.class)
            .handler(new ClientHandlersInitializer(TcpClient.this));
    }

    private ChannelFutureListener getConnectionListener() {
        return new ChannelFutureListener() {
            @Override
            public void operationComplete(ChannelFuture future) throws Exception
            {
                if (!future.isSuccess()) {
                    future.channel().pipeline().fireChannelInactive();
                }
            }
        };
    }

    public static void main(String[] args) {
        TcpClient tcpClient = new TcpClient("localhost", 2222);
        tcpClient.connect();
    }
}

```

在测试之前，为了避开 Connection reset by peer 异常，可以稍微修改Pinger的ping()方法，添加if (second == 5)的条件判断。如下：

```

private void ping(Channel channel) {
    int second = Math.max(1, random.nextInt(baseRandom));
    if (second == 5) {
        second = 6;
    }
    System.out.println("next heart beat will send after " + second + "s.");
}

```

```

        ScheduledFuture<?> future = channel.eventLoop().schedule(new Runnable()
{
    @Override
    public void run() {
        if (channel.isActive()) {
            System.out.println("sending heart beat to the server...");
            channel.writeAndFlush(ClientIdleStateTrigger.HEART_BEAT);
        } else {
            System.err.println("The connection had broken, cancel the
task that will send a heart beat.");
            channel.closeFuture();
            throw new RuntimeException();
        }
    }
}, second, TimeUnit.SECONDS);

future.addListener(new GenericFutureListener() {
    @Override
    public void operationComplete(Future future) throws Exception {
        if (future.isSuccess()) {
            ping(channel);
        }
    }
});
}

```

启动客户端

先只启动客户端，观察控制台输出，可以看到类似如下日志：

```
15:20:58.441 [main] DEBUG io.netty.buffer.ByteBufUtil - -Dio.netty.th
15:20:58.441 [main] DEBUG io.netty.buffer.ByteBufUtil - -Dio.netty.ma
Try to reconnect to the server after 1000ms. Retry count: 1.
Lost the TCP connection with the server.
15:20:58.585 [nioEventLoopGroup-2-1] DEBUG io.netty.util.Recycler - -
15:20:58.585 [nioEventLoopGroup-2-1] DEBUG io.netty.util.Recycler - -
15:20:58.586 [nioEventLoopGroup-2-1] DEBUG io.netty.util.Recycler - -
15:20:58.586 [nioEventLoopGroup-2-1] DEBUG io.netty.util.Recycler - -
Reconnecting ...
Try to reconnect to the server after 1000ms. Retry count: 2.
Reconnecting ...
Try to reconnect to the server after 1000ms. Retry count: 3.
Reconnecting ...
Try to reconnect to the server after 3000ms. Retry count: 4.
Reconnecting ...
Try to reconnect to the server after 2000ms. Retry count: 5.
Reconnecting ...
Try to reconnect to the server after 1000ms. Retry count: 6.
Reconnecting ...
Try to reconnect to the server after 34000ms. Retry count: 7.
Reconnecting ...
Sleep extension too large (120000). Pinning to 60000
Try to reconnect to the server after 60000ms. Retry count: 8.
Reconnecting ...
Sleep extension too large (156000). Pinning to 60000
Try to reconnect to the server after 60000ms. Retry count: 9.
Reconnecting ...
Sleep extension too large (488000). Pinning to 60000
Try to reconnect to the server after 60000ms. Retry count: 10.
```

断线重连测试——客户端控制台输出

可以看到，当客户端发现无法连接到服务器端，所以一直尝试重连。随着重试次数增加，重试时间间隔越大，但又不想无限增大下去，所以需要定一个阈值，比如60s。如上图所示，当下一次重试时间超过60s时，会打印Sleep extension too large(*). Pinning to 60000，单位为ms。出现这句话的意思是，计算出来的时间超过阈值（60s），所以把真正睡眠的时间重置为阈值（60s）。

聊聊：Netty 如何实现心跳机制？

何为心跳

所谓心跳，即在 TCP 长连接中，客户端和服务端之间定期发送的一种特殊的数据包，通知对方自己还在线，以确保 TCP 连接的有效性。

注：心跳包还有另一个作用，经常被忽略，即：一个连接如果长时间不用，防火墙或者路由器就会断开该连接。

如何实现

核心Handler —— IdleStateHandler

在 Netty 中，实现心跳机制的关键是 IdleStateHandler，那么这个 Handler 如何使用呢？先看下它的构造器：

```
public IdleStateHandler(int readerIdleTimeSeconds, int writerIdleTimeSeconds,
int allIdleTimeSeconds) {
    this((long)readerIdleTimeSeconds, (long)writerIdleTimeSeconds,
(long)allIdleTimeSeconds, TimeUnit.SECONDS);
}
```

这里解释下三个参数的含义：

- readerIdleTimeSeconds: 读超时. 即当在指定的时间间隔内没有从 Channel 读取到数据时, 会触发一个 READER_IDLE 的 IdleStateEvent 事件.
- writerIdleTimeSeconds: 写超时. 即当在指定的时间间隔内没有数据写入到 Channel 时, 会触发一个 WRITER_IDLE 的 IdleStateEvent 事件.
- allIdleTimeSeconds: 读/写超时. 即当在指定的时间间隔内没有读或写操作时, 会触发一个 ALL_IDLE 的 IdleStateEvent 事件.

注：这三个参数默认的时间单位是秒。若需要指定其他时间单位，可以使用另一个构造方法：
IdleStateHandler(boolean observeOutput, long readerIdleTime, long writerIdleTime, long allIdleTime, TimeUnit unit)

在看下面的实现之前，建议先了解一下IdleStateHandler的实现原理。

下面直接上代码，需要注意的地方，会在代码中通过注释进行说明。

使用IdleStateHandler实现心跳

下面将使用IdleStateHandler来实现心跳，Client端连接到Server端后，会循环执行一个任务：**随机等待几秒，然后ping一下Server端，即发送一个心跳包**。当等待的时间超过规定时间，将会发送失败，以为Server端在此之前已经主动断开连接了。代码如下：

Client端

ClientIdleStateTrigger —— 心跳触发器

类ClientIdleStateTrigger也是一个Handler，只是重写了userEventTriggered方法，用于捕获IdleState.WRITER_IDLE事件（未在指定时间内向服务器发送数据），然后向Server端发送一个心跳包。

```
/**
 * <p>
 * 用于捕获{@link IdleState#WRITER_IDLE}事件（未在指定时间内向服务器发送数据），然后向
 <code>Server</code>端发送一个心跳包。
 * </p>
 */
public class ClientIdleStateTrigger extends ChannelInboundHandlerAdapter {

    public static final String HEART_BEAT = "heart beat!";

    @Override
    public void userEventTriggered(ChannelHandlerContext ctx, Object evt) throws
Exception {
        if (evt instanceof IdleStateEvent) {
            IdleState state = ((IdleStateEvent) evt).state();
            if (state == IdleState.WRITER_IDLE) {
                // write heartbeat to server
                ctx.writeAndFlush(HEART_BEAT);
            }
        } else {
            super.userEventTriggered(ctx, evt);
        }
    }
}
```

```

    }
}

}

```

Pinger —— 心跳发射器

```

/**
 * <p>客户端连接到服务器端后，会循环执行一个任务：随机等待几秒，然后ping一下Server端，即发送
 * 一个心跳包。</p>
 */
public class Pinger extends ChannelInboundHandlerAdapter {

    private Random random = new Random();
    private int baseRandom = 8;

    private Channel channel;

    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        super.channelActive(ctx);
        this.channel = ctx.channel();

        ping(ctx.channel());
    }

    private void ping(Channel channel) {
        int second = Math.max(1, random.nextInt(baseRandom));
        System.out.println("next heart beat will send after " + second + "s.");
        ScheduledFuture<?> future = channel.eventLoop().schedule(new Runnable()
        {
            @Override
            public void run() {
                if (channel.isActive()) {
                    System.out.println("sending heart beat to the server...");
                    channel.writeAndFlush(ClientIdleStateTrigger.HEART_BEAT);
                } else {
                    System.err.println("The connection had broken, cancel the
task that will send a heart beat.");
                    channel.closeFuture();
                    throw new RuntimeException();
                }
            }
        }, second, TimeUnit.SECONDS);

        future.addListener(new GenericFutureListener() {
            @Override
            public void operationComplete(Future future) throws Exception {
                if (future.isSuccess()) {
                    ping(channel);
                }
            }
        });
    }

    @Override

```

```

        public void exceptionCaught(ChannelHandlerContext ctx, Throwable cause)
        throws Exception {
            // 当Channel已经断开的情况下，仍然发送数据，会抛异常，该方法会被调用。
            cause.printStackTrace();
            ctx.close();
        }
    }
}

```

ClientHandlersInitializer —— 客户端处理器集合的初始化类

```

public class ClientHandlersInitializer extends ChannelInitializer<SocketChannel>
{

    private ReconnectHandler reconnectHandler;
    private EchoHandler echoHandler;

    public ClientHandlersInitializer(TcpClient tcpClient) {
        Assert.notNull(tcpClient, "TcpClient can not be null.");
        this.reconnectHandler = new ReconnectHandler(tcpClient);
        this.echoHandler = new EchoHandler();
    }

    @Override
    protected void initChannel(SocketChannel ch) throws Exception {
        ChannelPipeline pipeline = ch.pipeline();
        pipeline.addLast(new LengthFieldBasedFrameDecoder(Integer.MAX_VALUE, 0,
4, 0, 4));
        pipeline.addLast(new LengthFieldPrepender(4));
        pipeline.addLast(new StringDecoder(CharsetUtil.UTF_8));
        pipeline.addLast(new StringEncoder(CharsetUtil.UTF_8));
        pipeline.addLast(new Pinger());
    }
}

```

注：上面的Handler集合，除了Pinger，其他都是编解码器和解决粘包，可以忽略。

TcpClient —— TCP连接的客户端

```

public class TcpClient {

    private String host;
    private int port;
    private Bootstrap bootstrap;
    /** 将<code>Channel</code>保存起来，可用于在其他非handler的地方发送数据 */
    private Channel channel;

    public TcpClient(String host, int port) {
        this(host, port, new ExponentialBackOffRetry(1000, Integer.MAX_VALUE, 60
* 1000));
    }

    public TcpClient(String host, int port, RetryPolicy retryPolicy) {
        this.host = host;
        this.port = port;
        init();
    }
}

```

```

/**
 * 向远程TCP服务器请求连接
 */
public void connect() {
    synchronized (bootstrap) {
        ChannelFuture future = bootstrap.connect(host, port);
        this.channel = future.channel();
    }
}

private void init() {
    EventLoopGroup group = new NioEventLoopGroup();
    // bootstrap 可重用，只需在TcpClient实例化的时候初始化即可。
    bootstrap = new Bootstrap();
    bootstrap.group(group)
        .channel(NioSocketChannel.class)
        .handler(new ClientHandlersInitializer(TcpClient.this));
}

public static void main(String[] args) {
    TcpClient tcpClient = new TcpClient("localhost", 2222);
    tcpClient.connect();
}
}

```

Server端

ServerIdleStateTrigger —— 断连触发器

```

/**
 * <p>在规定时间内未收到客户端的任何数据包，将主动断开该连接</p>
 */
public class ServerIdleStateTrigger extends ChannelInboundHandlerAdapter {
    @Override
    public void userEventTriggered(ChannelHandlerContext ctx, Object evt) throws
    Exception {
        if (evt instanceof IdleStateEvent) {
            IdleState state = ((IdleStateEvent) evt).state();
            if (state == IdleState.READER_IDLE) {
                // 在规定时间内没有收到客户端的上行数据，主动断开连接
                ctx.disconnect();
            }
        } else {
            super.userEventTriggered(ctx, evt);
        }
    }
}

```

ServerBizHandler —— 服务器端的业务处理器

```

/**
 * <p>收到来自客户端的数据包后，直接在控制台打印出来.</p>
 */
@ChannelHandler.Sharable
public class ServerBizHandler extends SimpleChannelInboundHandler<String> {

```

```

        private final String REC_HEART_BEAT = "I had received the heart beat!";

        @Override
        protected void channelRead0(ChannelHandlerContext ctx, String data) throws
Exception {
            try {
                System.out.println("receive data: " + data);
                //          ctx.writeAndFlush(REC_HEART_BEAT);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }

        @Override
        public void channelActive(ChannelHandlerContext ctx) throws Exception {
            System.out.println("Established connection with the remote client.");

            // do something

            ctx.fireChannelActive();
        }

        @Override
        public void channelInactive(ChannelHandlerContext ctx) throws Exception {
            System.out.println("Disconnected with the remote client.");

            // do something

            ctx.fireChannelInactive();
        }

        @Override
        public void exceptionCaught(ChannelHandlerContext ctx, Throwable cause)
throws Exception {
            cause.printStackTrace();
            ctx.close();
        }
    }
}

```

ServerHandlerInitializer —— 服务器端处理器集合的初始化类

```

/**
 * <p>用于初始化服务器端涉及到的所有<code>Handler</code></p>
 */
public class ServerHandlerInitializer extends ChannelInitializer<SocketChannel>
{

    protected void initChannel(SocketChannel ch) throws Exception {
        ch.pipeline().addLast("idleStateHandler", new IdleStateHandler(5, 0,
0));
        ch.pipeline().addLast("idleStateTrigger", new ServerIdleStateTrigger());
        ch.pipeline().addLast("frameDecoder", new
LengthFieldBasedFrameDecoder(Integer.MAX_VALUE, 0, 4, 0, 4));
        ch.pipeline().addLast("frameEncoder", new LengthFieldPrepender(4));
        ch.pipeline().addLast("decoder", new StringDecoder());
        ch.pipeline().addLast("encoder", new StringEncoder());
        ch.pipeline().addLast("bizHandler", new ServerBizHandler());
    }
}

```

```
}  
  
}
```

注：new IdleStateHandler(5, 0, 0)该handler代表如果在5秒内没有收到来自客户端的任何数据包（包括但不限于心跳包），将会主动断开与该客户端的连接。

TcpServer —— 服务器端

```
public class TcpServer {  
    private int port;  
    private ServerHandlerInitializer serverHandlerInitializer;  
  
    public TcpServer(int port) {  
        this.port = port;  
        this.serverHandlerInitializer = new ServerHandlerInitializer();  
    }  
  
    public void start() {  
        EventLoopGroup bossGroup = new NioEventLoopGroup(1);  
        EventLoopGroup workerGroup = new NioEventLoopGroup();  
        try {  
            ServerBootstrap bootstrap = new ServerBootstrap();  
            bootstrap.group(bossGroup, workerGroup)  
                .channel(NioServerSocketChannel.class)  
                .childHandler(this.serverHandlerInitializer);  
            // 绑定端口，开始接收进来的连接  
            ChannelFuture future = bootstrap.bind(port).sync();  
  
            System.out.println("Server start listen at " + port);  
            future.channel().closeFuture().sync();  
        } catch (Exception e) {  
            bossGroup.shutdownGracefully();  
            workerGroup.shutdownGracefully();  
            e.printStackTrace();  
        }  
    }  
  
    public static void main(String[] args) throws Exception {  
        int port = 2222;  
        new TcpServer(port).start();  
    }  
}
```

至此，所有代码已经编写完毕。

测试

首先启动客户端，再启动服务器端。启动完成后，在客户端的控制台上，可以看到打印如下类似日志：

```

14:08:08.600 [main] DEBUG io.netty.buffer.PooledByteBufAllocator - ~Dio.netty.allocator.maxCachedBufferCapacity: 32768
14:08:08.600 [main] DEBUG io.netty.buffer.PooledByteBufAllocator - ~Dio.netty.allocator.cacheTrimInterval: 8192
14:08:08.600 [main] DEBUG io.netty.buffer.PooledByteBufAllocator - ~Dio.netty.allocator.useCacheForAllThreads: true
14:08:08.617 [main] DEBUG io.netty.buffer.ByteBufUtil - ~Dio.netty.allocator.type: pooled
14:08:08.617 [main] DEBUG io.netty.buffer.ByteBufUtil - ~Dio.netty.threadLocalDirectBufferSize: 65536
14:08:08.617 [main] DEBUG io.netty.buffer.ByteBufUtil - ~Dio.netty.maxThreadLocalCharBufferSize: 16384
next heart beat will send after 2s.
sending heart beat to the server...
14:08:10.790 [nioEventLoopGroup-2-1] DEBUG io.netty.util.Recycler - ~Dio.netty.recycler.maxCapacityPerThread: 32768
14:08:10.790 [nioEventLoopGroup-2-1] DEBUG io.netty.util.Recycler - ~Dio.netty.recycler.maxSharedCapacityFactor: 2
14:08:10.790 [nioEventLoopGroup-2-1] DEBUG io.netty.util.Recycler - ~Dio.netty.recycler.linkCapacity: 16
14:08:10.790 [nioEventLoopGroup-2-1] DEBUG io.netty.util.Recycler - ~Dio.netty.recycler.ratio: 8
14:08:10.825 [nioEventLoopGroup-2-1] DEBUG io.netty.buffer.AbstractByteBuf - ~Dio.netty.buffer.bytebuf.checkAccessible: true
14:08:10.827 [nioEventLoopGroup-2-1] DEBUG io.netty.util.ResourceLeakDetectorFactory - Loaded default ResourceLeakDetector: io.netty.
next heart beat will send after 2s.
sending heart beat to the server...
next heart beat will send after 3s.
sending heart beat to the server...
next heart beat will send after 2s.
sending heart beat to the server...
next heart beat will send after 6s.
The connection had broken, cancel the task that will send a heart beat.

```

下一个心跳包会在6s后发送，但在第5s的时候，服务器端因为长时间未收到客户端的数据包已经把连接断开，所以在第6s发送数据包的时候就发现连接已断开，心跳包发送失败，然后打印了下面那条日志。

客户端控制台输出的日志

在服务器端可以看到控制台输出了类似如下的日志：

```

receive data: heart beat!
receive data: heart beat!
receive data: heart beat!
receive data: heart beat!
Disconnected with the remote client.

```

服务器端控制台输出的日志

默认情况 Netty 起多少线程？何时启动？

如果你在简历上写了Netty，那么面试官百分之九十的可能会问你Netty默认其多少线程？在什么时候启动的问题。

面试官一方面是想考验你对Netty有没有最基本的知识点掌握，一方面是想试探你有没有深入了解过Netty的[源码](#)和启动流程。

你在编写Netty服务端的时候经常会编写下面的代码：

```

EventLoopGroup boss = new NioEventLoopGroup();
EventLoopGroup worker = new NioEventLoopGroup();

```

这里构建了两个事件循环组，循环组具体是干什么的我就暂时不细说了。我们拿其中的一个来进行分析底层的源码即可。这里调用了无参的构造方法，也就使用了默认的线程数，那么默认线程数是多少呢？慢慢进行分析：

```

public NioEventLoopGroup() {
    this(0);
}

```

调用了“this(0)”，也就是调用了单个参数的构造方法，然后调用的传递线程数是0。

```
public NioEventLoopGroup(int nThreads) {
    this(nThreads, (Executor) null);
}
```

紧接着调用 线程数和线程池接口的方法，需要注意的是，这里的“executor”是null。

```
public NioEventLoopGroup(int nThreads, Executor executor) {
    this(nThreads, executor, SelectorProvider.provider());
}
```

因为Netty底层封装了NIO所以调用了传递“provider”的构造参数。

```
public NioEventLoopGroup(int nThreads, Executor executor, final SelectorProvider
selectorProvider) {
    this(nThreads, executor, selectorProvider,
DefaultSelectStrategyFactory.INSTANCE);
}
```

紧接着比之前的多了一个默认选择策略工厂，传递了一个单例对象。

```
public NioEventLoopGroup(int nThreads, Executor executor, final SelectorProvider
selectorProvider, final SelectStrategyFactory selectStrategyFactory) {
    super(nThreads, executor, selectorProvider, selectStrategyFactory,
RejectedExecutionHandlers.reject());
}
```

之后调用了父类的构造方法，这里又比之前多了一个参数是异常拒绝处理器，也就是出现异常的时候执行的拒绝处理器。

```
protected MultithreadEventLoopGroup(int nThreads, Executor executor, Object...
args) {
    super(nThreads == 0 ? DEFAULT_EVENT_LOOP_THREADS : nThreads, executor,
args);
}
```

紧接着又调用了父类的“MultithreadEventLoopGroup”构造方法，下面这个就是默认初始化的线程了：

```
nThreads == 0 ? DEFAULT_EVENT_LOOP_THREADS : nThreads
```

这里是一个三元表达式，判断传递过来的参数“nThreads”是否等于0，如果是的话就赋值“DEFAULT_EVENT_LOOP_THREADS”，如果不是的话就赋值为传递过来的值。

那么“DEFAULT_EVENT_LOOP_THREADS”是在什么时候初始化的呢？是在静态代码块中初始化。

```
private static final int DEFAULT_EVENT_LOOP_THREADS;
static {
    DEFAULT_EVENT_LOOP_THREADS = Math.max(1, SystemPropertyUtil.getInt(
        "io.netty.eventLoopThreads", NettyRuntime.availableProcessors() *
2));

    if (logger.isDebugEnabled()) {
        logger.debug("-Dio.netty.eventLoopThreads: {}",
DEFAULT_EVENT_LOOP_THREADS);
    }
}
```

它获取的值也就是“`NettyRuntime.availableProcessors() * 2`”。

然后调用了“`availableProcessors`”的方法：

```
public static int availableProcessors()
{
    return holder.availableProcessors();
}

synchronized int availableProcessors() {
    if (this.availableProcessors == 0) {
        final int availableProcessors =SystemPropertyUtil.getInt(
            "io.netty.availableProcessors",
            Runtime.getRuntime().availableProcessors());
        setAvailableProcessors(availableProcessors);
    }
    return this.availableProcessors;
}
```

需要注意的就是“`Runtime.getRuntime().availableProcessors()`”这个方法，这个方法的意思是获取运行时可用线程数。接下来写个方法测试一下：

```
public class NettyServer {
    public static void main(String[] args) {
        System.out.println(Runtime.getRuntime().availableProcessors());
    }
}
```

由此可见，Netty的默认启动了电脑可用线程数的两倍，在调用了bind方法的时候执行。

“`Runtime.getRuntime().availableProcessors()`”

那么反过来思考下！

我们真的需要每个EventLoopGroup都默认启用线程吗？

答案当然不是！

我们具体的线程数应该交由业务来决定而不是使用默认的参数。

有的时候我们只需要使用一个线程来监听注册事件。

比如，在客户端代码中，线程数的调整，非常有必要。

聊聊：NioEventLoopGroup 默认的构造函数会起多少线程？

同上

聊聊：如何设计一个内存池，或者内存分配器

这是出现的次数比较多的大厂面试题：怎样设计一个内存池和内存分配器？

尼恩暗语：会的人不多，如果能答上来，面试官会两眼发绿光

重要性：对内存分配器透彻理解，并且具备实操能力，是编程高手的标志之一。

如果你不能理解malloc之类内存分配器实现原理的话，那你就可能写不出高性能程序，写不出高性能程序就很难参与核心项目，参与不了核心项目那么很难升职加薪，很难升职加薪就无法走向人生巅峰，所以，内存分配非常关键

说明：此题是一个实操性质的题目，后续尼恩带大家参考netty内存池，从0到1，架构、设计、实现一个高性能内存池组件

关于对象池的知识，请参见[Netty内存池（史上最全 + 5W字长文）](#)

聊聊：如何设计一个Java对象池，减少GC和内存分配消耗

重要性：对对象池透彻理解，并且具备实操能力，也是编程高手的标志之一。

对象池顾名思义就是存放对象的池，与我们常听到的线程池、数据库连接池、http连接池等一样，都是典型的池化设计思想。

对象池的优点就是可以集中管理池中对象，减少频繁创建和销毁长期使用的对象，从而提升复用性，以节约资源的消耗，可以有效避免频繁为对象分配内存和释放堆中内存，进而减轻jvm垃圾收集器的负担，避免内存抖动。

Apache Common Pool2 是Apache提供的一个通用对象池技术实现，可以方便定制化自己需要的对象池，大名鼎鼎的 Redis 客户端 Jedis 内部连接池就是基于它来实现的。

说明：此题是一个实操性质的题目，后续尼恩带大家参考netty对象池，从0到1，架构、设计、实现一个高性能对象池组件

美团二面：epoll性能那么高，为什么？

说在前面

在40岁老架构师 尼恩的[读者交流群](#)(50+)中，最近有小伙伴拿到了一线互联网企业如美团、拼多多、极兔、有赞、希音的面试资格，遇到一几个很重要的面试题：

- 说说epoll的数据结构
- 说说epoll的实现原理
- 协议栈如何与epoll通信？
- epoll线程安全如何加锁？
- 说说ET与LT的实现
-

这里尼恩给大家做一下系统化、体系化的梳理，使得大家可以充分展示一下大家雄厚的“技术肌肉”，让面试官爱到“不能自己、口水直流”。

也一并把这个题目以及参考答案，收入咱们的《[尼恩Java面试宝典](#)》V88版本，供后面的小伙伴参考，提升大家的 3高 架构、设计、开发水平。

最新《尼恩 架构笔记》《尼恩高并发三部曲》《尼恩Java面试宝典》的PDF文件，请关注公众号【技术自由圈】领取，暗号：领电子书

epoll的数据结构

多种数据结构进行决策

epoll至少需要两个集合

- 所有fd的总集
- 就绪fd的集合

那么这个总集选用什么数据结构存储呢？

我们知道，一个fd，其底层对应一个TCB。那么也就是说key=fd,val=TCB,是一个典型的kv型数据结构，对于kv型数据结构我们可以使用以下三种进行存储。

1. hash
2. 红黑树
3. b/b+tree

如果使用hash进行存储，其优点是查询速度很快， $O(1)$ 。

但是在我们调用epoll_create()的时候，hash底层的数组创建多大合适呢？

如果有百万的fd，那么这个数组越大越好，如果我们仅仅十几个fd需要管理，在创建数组的时候，太大的空间就很浪费。而这个fd我们又不能预先知道有多少，所以hash是不合适的。

b/b+tree是多叉树，一个结点可以存多个key，主要是用于降低层高，用于磁盘索引的，所以在我们这个内存场景下也是不适合的。

在内存索引的场景下我们一般使用红黑树来作为首选的数据结构，首先红黑树的查找速度很快， $O(\log(N))$ 。其次在调用epoll_create()的时候，只需要创建一个红黑树树根即可，无需浪费额外的空间。

那么就就绪集合用什么数据结构呢，首先就绪集合不是以查找为主的，就绪集合的作用是将里面的元素拷贝给用户进行处理，所以集合里的元素没有优先级，那么就可以采用线性的数据结构，使用队列来存储，先进先出，先就绪的先处理。

所有fd的总集 -----> 红黑树

就绪fd的集合 -----> 队列

红黑树和就绪队列的关系

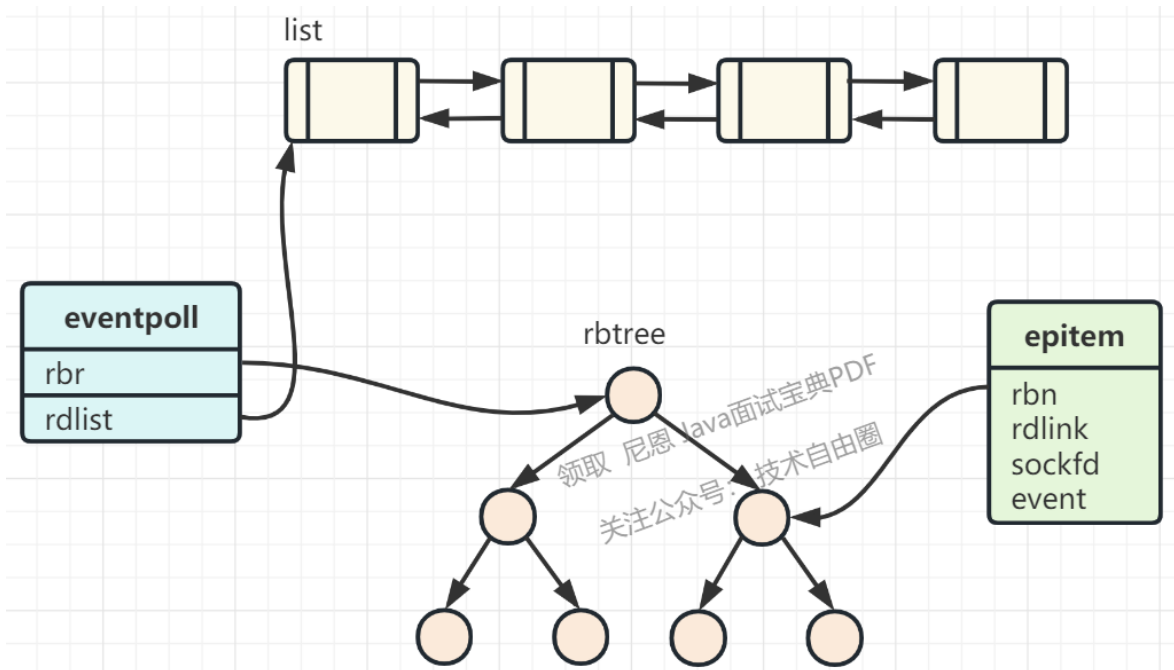
红黑树的结点和就绪队列的结点的同一个节点，所谓的加入就绪队列，就是将结点的前后指针联系在一起。所以就绪了不是将红黑树结点delete掉然后加入队列。他们是同一个结点，不需要delete。

```
struct epitem{
    RB_ENTRY(epitem) rbn;
    LIST_ENTRY(epitem) rdlink;
    int rdy; //exist in list

    int sockfd;
    struct epoll_event event;
};

struct eventpoll {
    ep_rb_tree rbr;
    int rbcnt;
    LIST_HEAD(,epitem) rdlist;
    int rdnum;

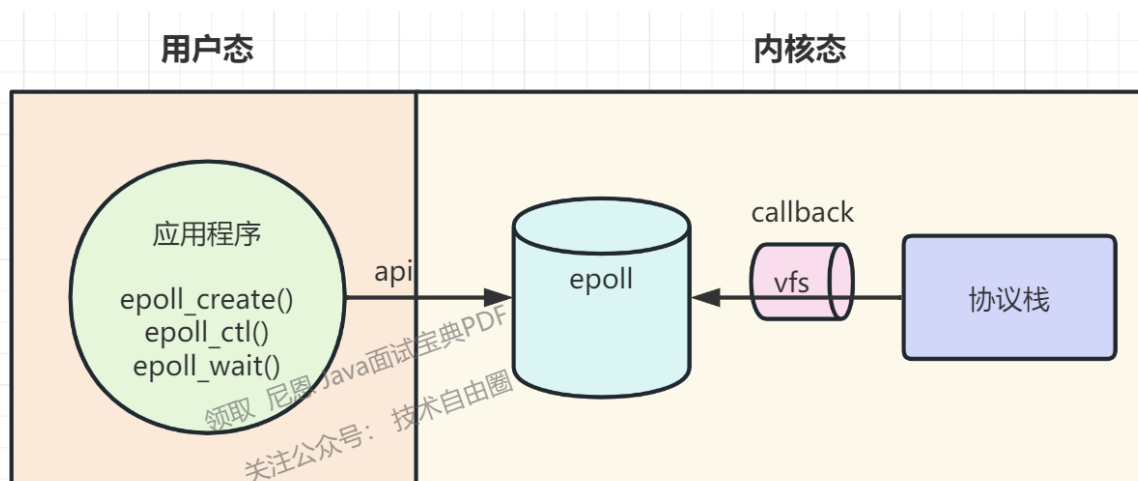
    int waiting;
    pthread_mutex_t mtx; //rbtree update
    pthread_spinlock_t lock; //rdlist update
    pthread_cond_t cond; //block for event
    pthread_mutex_t cdmtx; //mutex for cond
};
```



协议栈如何与epoll模块通信

epoll的工作环境

应用程序只能通过三个api接口来操作epoll。当一个io准备就绪的时候，epoll是怎么知道io准备就绪了呢？是由协议栈将数据解析出来触发回调通知epoll的。也就是说可以把epoll的工作环境看出三部分，左边应用程序的api，中间的epoll，右边是协议栈的回调(协议栈当然不能直接操作epoll，中间的vfs在此不是重点，就直接省略vfs这一层了)。



协议栈触发回调通知epoll的时机

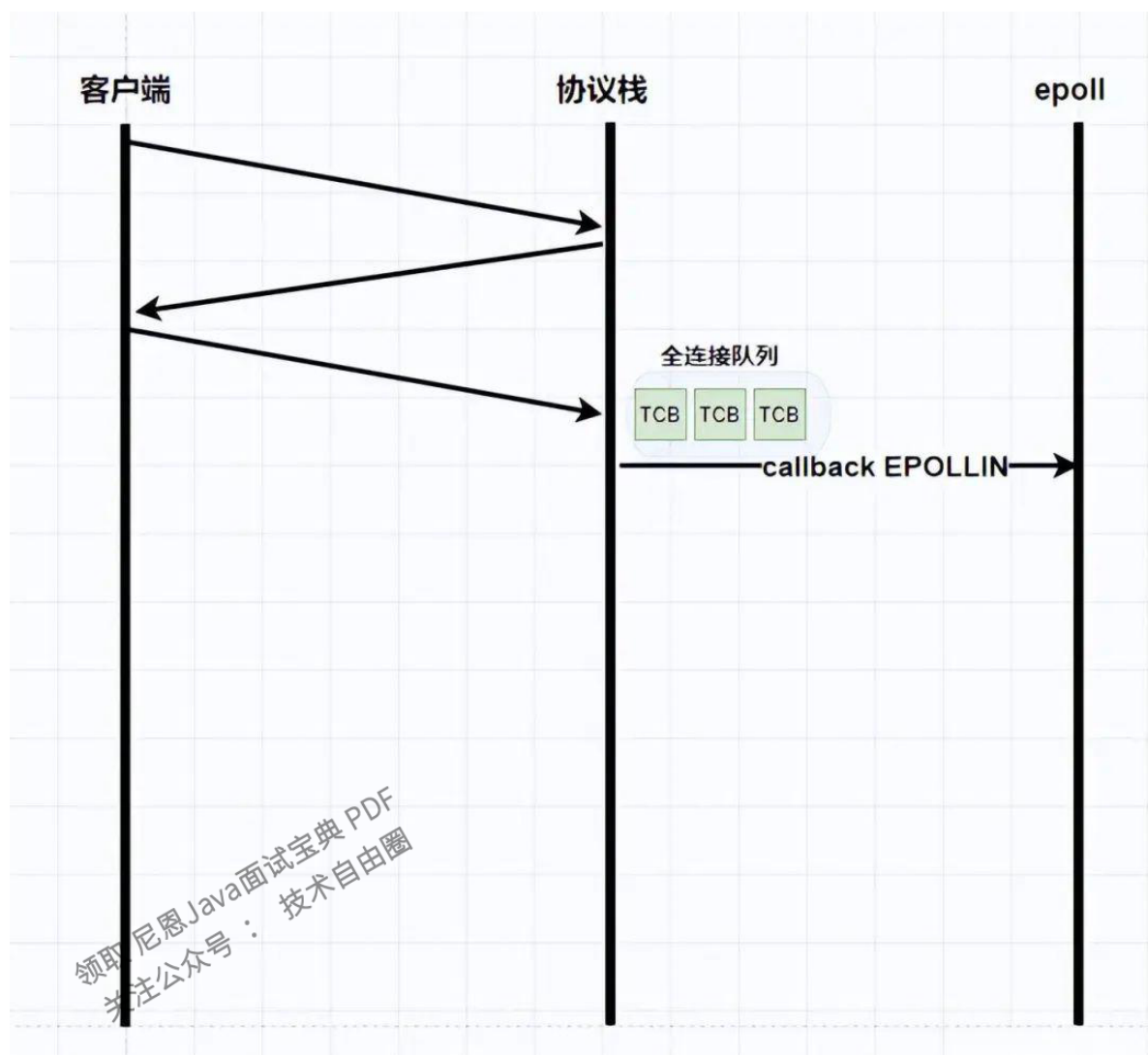
socket有两类，一类是监听listenfd，一类是客户端clientfd。对于sockfd而言，我们一般比较关注EPOLLIN和EPOLLOUT这两个事件，所以如果是listenfd，我们通常的做法就是accept。对于clientfd来说，如果可读我们就recv，如果可写我们就send。

协议栈将数据解析出来触发回调通知epoll。epoll是怎么知道哪个io就绪了呢？我们从ip头可以解析出源ip，目的ip和协议，从tcp头可以解析出源端口和目的端口，此时五元组就凑齐了。socket fd --- < 源IP地址, 源端口, 目的IP地址, 目的端口, 协议 > 一个fd就是一个五元组，知道了fd，我们就能从红黑树中找到对应的结点。

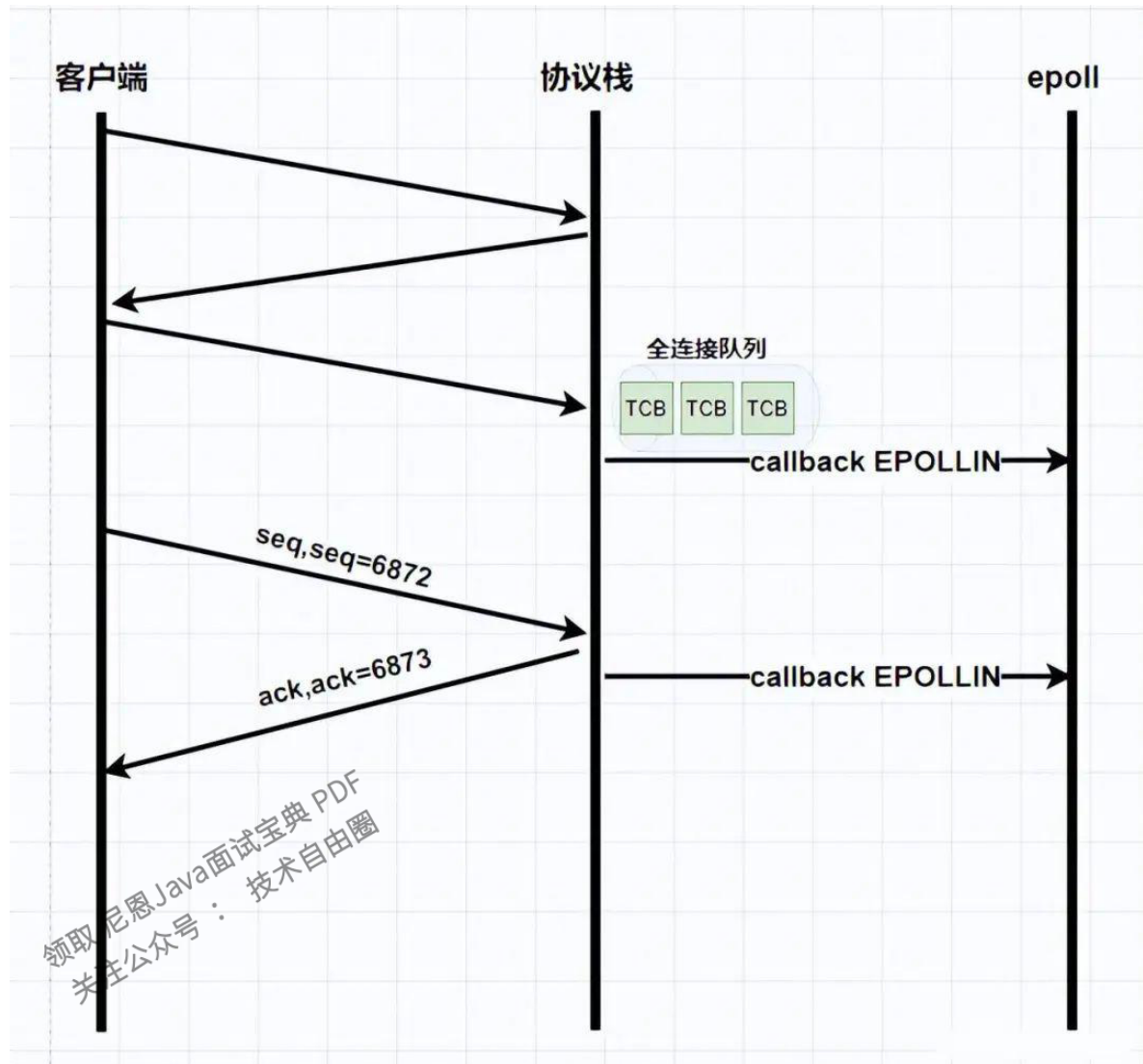
那么这个回调函数做什么事情呢？我们传入fd和具体事件这两个参数，然后做下面两个操作

1. 通过fd找到对应的结点
2. 把结点加入到就绪队列

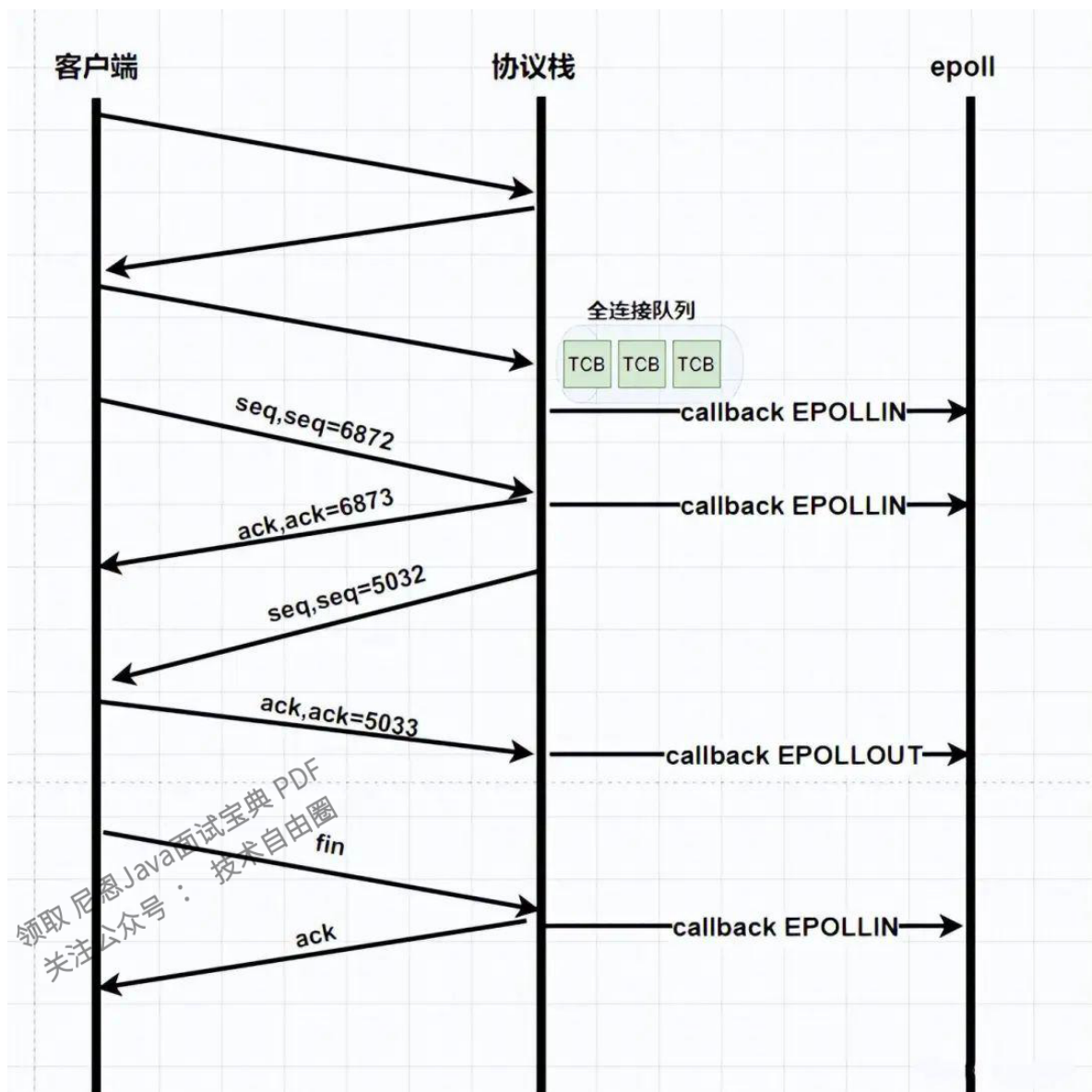
1、协议栈中，在三次握手完成之后，会往全连接队列中添加一个TCB结点，然后触发一个回调函数，通知到epoll里面有个EPOLLIN事件



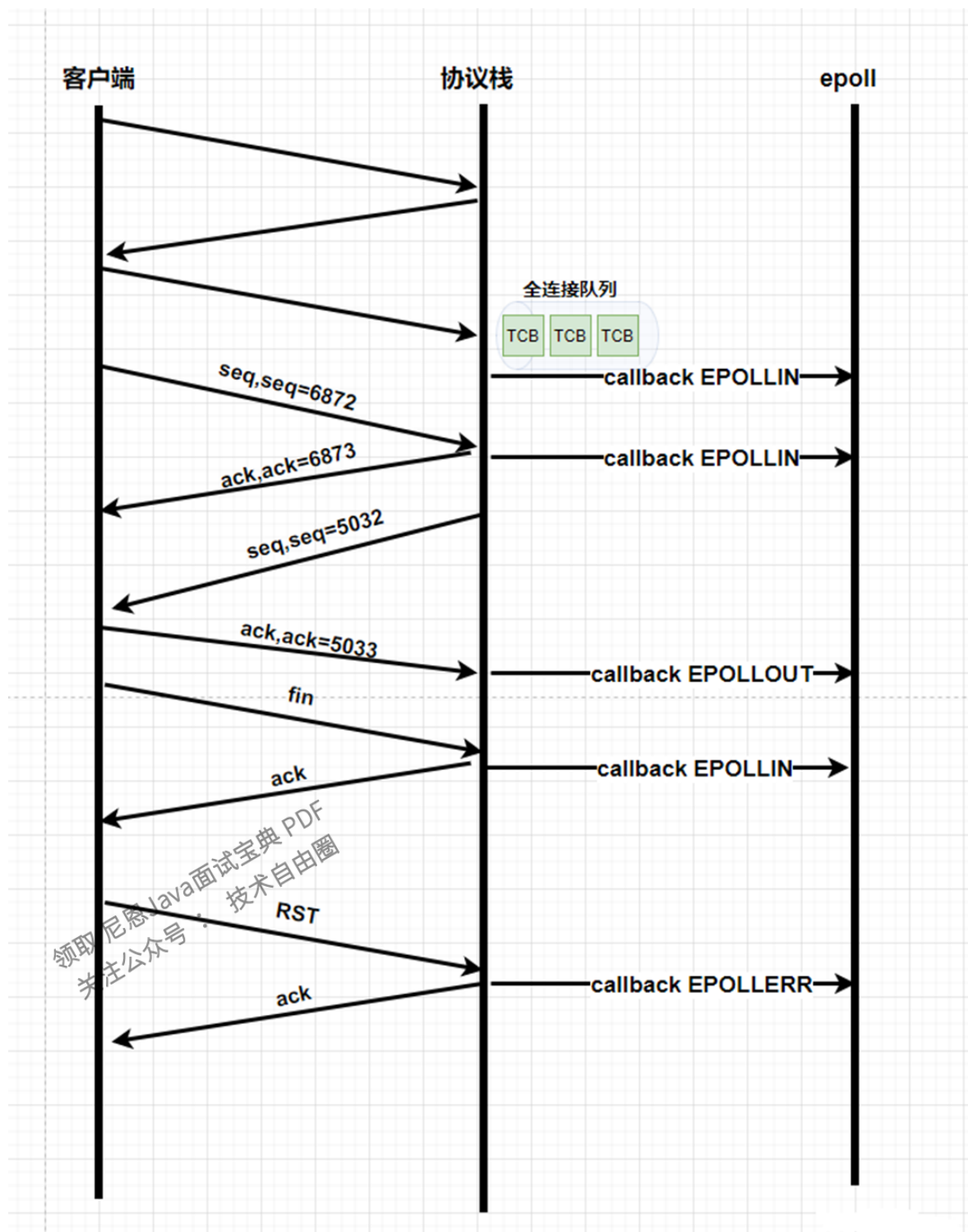
2、客户端发送一个数据包，协议栈接收后回复ACK，之后触发一个回调函数，通知到epoll里面有个EPOLLIN事件



3、每个连接的TCB里面都有一个sendbuf，在对端接收到数据并返回ACK以后，sendbuf就可以将这部分确认接收的数据清空，此时sendbuf里面就有剩余空间，此时触发一个回调函数，通知到epoll里面有个EPOLLOUT事件



5、当接收到rst标志位的时候，回复ack之后也会触发回调函数，通知epoll有一个EPOLLERR事件



通知的时机总结

一个有5个通知的地方

1. 三次握手完成之后,
2. 接收数据回复ACK之后,
3. 发送数据收到ACK之后,
4. 接收FIN回复ACK之后,
5. 接收RST回复ACK之后

从回调机制看epoll 与 select/poll的区别

由于select和poll没有本质的区别，所以下面统一称为poll。

```
// poll跟select类似，其实poll就是把select三个文件描述符集合变成一个集合了。

int select(int nfds, fd_set * readfds, fd_set *writefds, fd_set *exceptfds,
struct timeval *timeout);

int poll(struct pollfd *fds, nfds_t nfds, int timeout);
```

我们看到每次调用poll，都需要把总集fds拷贝到内核态，检测完之后，再有内核态拷贝的用户态，这就是poll。而epoll不是这样，epoll只要有新的io就调用epoll_ctl()加入到红黑树里面，一旦有触发就用epoll_wait()将有事件的结点带出来，可以看到他们的第一个区别：poll总是拷贝总集，如果有100w个fd，只有两三个就绪呢？这会造成大量资源浪费；而epoll总是将需要拷贝的东西进行拷贝，没有浪费。

第二个区别：我们从上面知道了epoll的事件都是由协议栈进行回调然后加入到就绪队列的，而poll呢？内核如何检测poll的io是否就绪？只能通过遍历的方法判断，所以poll检测io通过遍历的方法也是比较慢的。

所以两者的区别：

select/poll需要把总集copy到内核，而epoll不用

实现原理上面，select/poll 需要循环遍历总集是否有就绪，而epoll是那个结点就绪了就加入就绪队列里面。

注意：poll不一定就比epoll慢，在io量小的情况下，poll是比epoll快的，而在大io量下，epoll绝对是有主导地位的。至于有多少个io才算多，其实也很难说，一般认为500或者1024为分界点（我乱说的）。

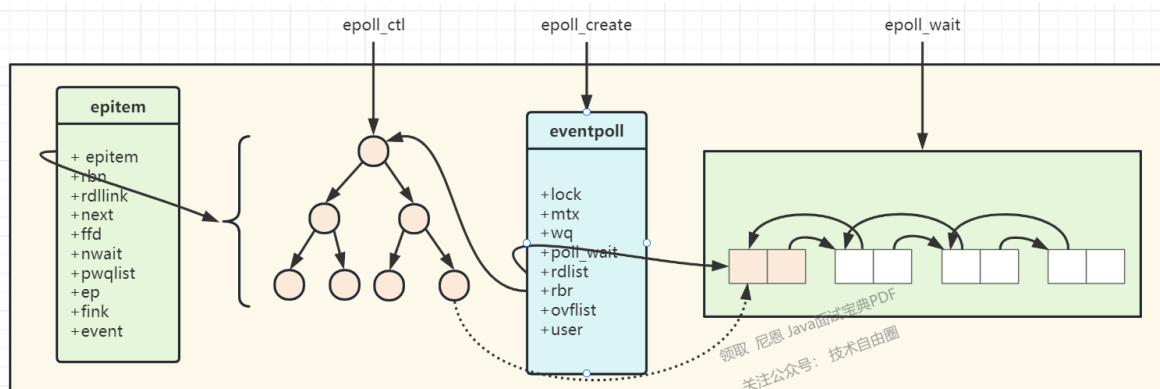
epoll线程安全如何加锁

3个api做什么事情

epoll_create() ==》创建红黑树的根节点

epoll_ctl() ==》add,del,mod 增加、删除、修改结点

epoll_wait() ==》把就绪队列的结点copy到用户态放到events里面，跟recv函数很像



分析加锁

如果有3个线程同时操作epoll，有哪些地方需要加锁？我们用户层面一共就只有3个api可以使用：

如果同时调用 `epoll_create()`，那就是创建三颗红黑树，没有涉及到资源竞争，没有关系。

如果同时调用 `epoll_ctl()`，对同一颗红黑树进行，增删改，这就涉及到资源竞争需要加锁了，此时我们对整棵树进行加锁。

如果同时调用 `epoll_wait()`，其操作的是就绪队列，所以需要对接就绪队列进行加锁。

我们要扣住epoll的工作环境，在应用程序调用 `epoll_ctl()`，协议栈会不会有回调操作红黑树结点？调用 `epoll_wait()` copy出来的时候，协议栈会不会操作红黑树结点加入就绪队列？综上所述：

```
epoll_ctl() 对红黑树加锁
epoll_wait() 对接就绪队列加锁
回调函数() 对红黑树加锁,对接就绪队列加锁
```

那么红黑树加什么锁，就绪队列加什么锁呢？

对于红黑树这种节点比较多的时候，采用互斥锁来加锁。

就绪队列就跟生产者消费者一样，结点是从协议栈回调函数来生产的，消费是 `epoll_wait()` 来消费。那么对于队列而言，用自旋锁（对于队列而言，插入删除比较简单，cpu自旋等待比让出的成本更低，所以用自旋锁）。

ET与LT如何实现

ET边沿触发，只触发一次

LT水平触发，如果没有读完就一直触发

代码如何实现ET和LT的效果呢？水平触发和边沿触发不是故意设计出来的，这是自然而然，水到渠成的功能。水平触发和边沿触发代码只需要改一点点就能实现。从协议栈检测到接收数据，就调用一次回调，这就是ET，接收到数据，调用一次回调。而LT水平触发，检测到 `recvbuf` 里面有数据就调用回调。所以ET和LT就是在使用回调的次数上面的差异。

那么具体如何实现呢？协议栈流程里面触发回调，是天然的符合ET只触发一次的。那么如果是LT，在 `recv` 之后，如果缓冲区还有数据那么加入到就绪队列。那么如果是LT，在 `send` 之后，如果缓冲区还有空间那么加入到就绪队列。那么这样就能实现LT了。

```

ssize_t nty_recv(int sockfd, char *buf, size_t len, int flags) {
    ...
    int event_remaining = 0;
    if (socket->epoll & NTY_EPOLLIN) {
        if (!(socket->epoll & NTY_EPOLLET) && rcv->recvbuf->merged_len > 0) {
            event_remaining = 1;
        }
    }
    if (event_remaining) {
        if (socket->epoll) {
            nty_epoll_add_event(tcp->ep, USR_SHADOW_EVENT_QUEUE, socket, NTY_EPOLLIN);
        }
    }
    ...
}

```

```

ssize_t nty_send(int sockfd, const char *buf, size_t len) {
    ...
    if (snd->snd_wnd > 0) {
        if ((socket->epoll & NTY_EPOLLOUT) && !(socket->epoll & NTY_EPOLLET)) {
            nty_epoll_add_event(tcp->ep, USR_SHADOW_EVENT_QUEUE, socket, NTY_EPOLLOUT);
        }
    }
    ...
}

```

(真实实现代码参看：linux-2.6.24/fs/eventpoll.c文件中的ep_send_events函数)

说在最后

Linux相关面试题，是非常常见的面试题。

以上的内容，如果大家能对答如流，如数家珍，基本上面试官会被你震惊到、吸引到。

最终，让面试官爱到“不能自己、口水直流”。offer，也就来了。

学习过程中，如果有啥问题，大家可以来找 40岁老架构师尼恩交流。

参考文献

<https://segmentfault.com/a/1190000022568931>

<https://blog.csdn.net/MonkeyBrothers/article/details/122860753>

<https://blog.csdn.net/kaihuishang666/article/details/116095480>

<http://www.wjhsh.net/hujinshui-p-10450548.html>

推荐阅读

《[百亿级访问量，如何做缓存架构设计](#)》

《[多级缓存 架构设计](#)》

《[消息推送 架构设计](#)》

《[阿里2面：你们部署多少节点？1000W并发，当如何部署？](#)》

《[美团2面：5个9高可用99.999%，如何实现？](#)》

《[网易一面：单节点2000Wtps，Kafka怎么做的？](#)》

《[字节一面：事务补偿和事务重试，关系是什么？](#)》

《[网易一面：25Wqps高吞吐写Mysql，100W数据4秒写完，如何实现？](#)》

《[亿级短视频，如何架构？](#)》

《[炸裂，靠“吹牛”过京东一面，月薪40K](#)》

《[太猛了，靠“吹牛”过顺丰一面，月薪30K](#)》

《[炸裂了...京东一面索命40问，过了就50W+](#)》

《[问麻了...阿里一面索命27问，过了就60W+](#)》

《[百度狂问3小时，大厂offer到手，小伙真狠！](#)》

《[饿了么太狠：面个高级Java，抖这多硬活、狠活](#)》

《[字节狂问一小时，小伙offer到手，太狠了！](#)》

《[收个滴滴Offer：从小伙三面经历，看看需要学点啥？](#)》

技术自由圈

实现架构转型，再无中年危机



关注**技术自由圈**公众号，获取每天技术干货

一起成为牛逼的**未来超级架构师**

几十篇架构笔记、5000页面试宝典、20个技术圣经

请加尼恩个人微信 免费拿走

暗号，请在 公众号后台 发送消息：**领电子书**

未来职业，如何突围：三栖架构师

未来职业，如何突围？

技术自由圈



——未来超级架构师社区

领路式指导

FSAC 三栖合一架构师

Future Super Architect Community

- 第一栖：Java 架构
- 第二栖：GO 架构
- 第三栖：大数据 架构

尼恩JAVA硬核架构班

会员制

提供技术方向指导，
职业生涯指导，少坑，少走弯路

简历指导

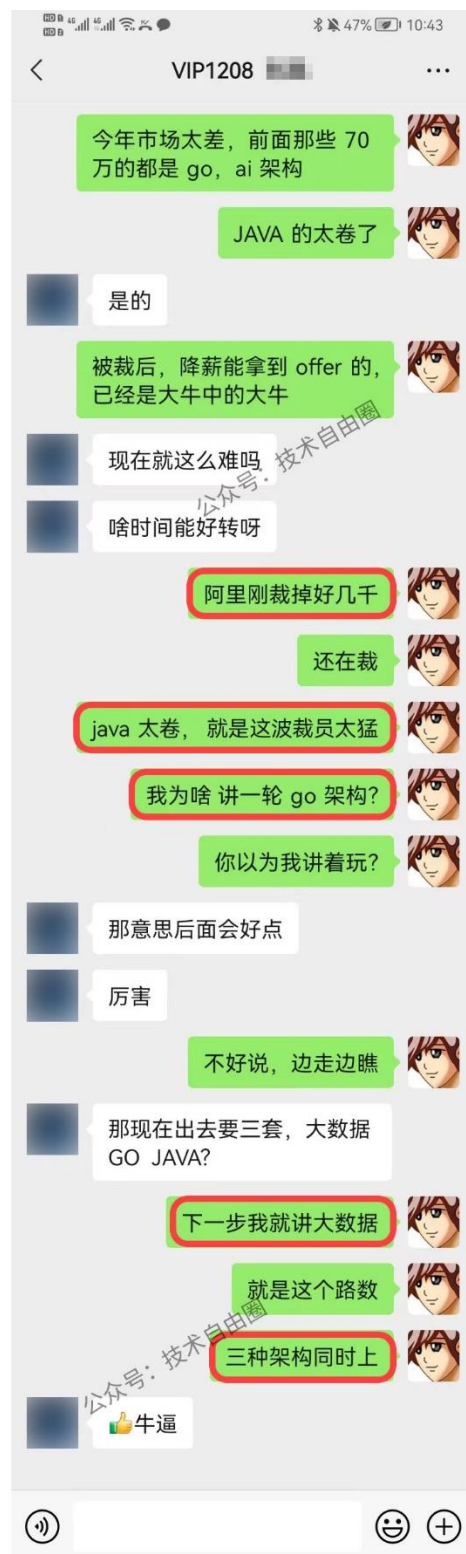
有助成功就业、跳槽大厂
挪窝涨薪必备

实操性

项目都是老架构师
在生产上实操过的项目

非水货

老架构师，不是水货架构师
《Java高并发三部曲》为证



成功案例：2年翻3倍，35岁卷王成功转型为架构师

详情：<http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新

最后发表

热门

精华

成功案例：[1057号卷王] 3年小伙拿到外企offer，薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例：[645号卷王] 4年经验卷王逆袭，被毕业后，反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例：[878号卷王] 小伙8年经验，年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例：通过尼恩的指导，小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例：[493号卷王] 5年小伙拿满意offer，就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例：[250号卷王] 就业极寒时代，收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例：[612号卷王] 就业极寒时代，从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例：[913号卷王] 热烈祝贺6年经验卷王，年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例：[959号卷王] 4年经验卷王，喜获百度、Boss直聘等N个优质offer，最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例：[529号卷王] 5年经验卷王喜收2大offer，最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例：[811号卷王] 热烈祝贺7年经验卷王，薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例：[287号卷王] 不惧大寒潮，卷王逆市收4 offer，涨30%，可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例：[1002号卷王] 5月份“被毕业”，改简历后，斩获顶级央企Offer，涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

1 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

1 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

1 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

1 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

1 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

1 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

1 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

1 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

1 卷王1号 超级版主 2022-3-25

❑ 成功案例: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

① 卷王1号 超级版主 2022-3-24

❑ 成功案例: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

① 卷王1号 超级版主 2022-3-21

❑ 成功案例: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

① 卷王1号 超级版主 2022-3-20

❑ 成功案例: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

① 卷王1号 超级版主 2022-3-16

❑ 简历案例: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 祝贺我圈两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

尼恩 超级版主 2022-3-5

❑ 成功案例: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂Offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [449号卷王] 应届毕业卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

❑ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

硬核推荐：尼恩Java硬核架构班

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>

尼恩Java 硬核架构班

已经发布

- ★ 《高性能RPC的基础实操之：从0到1开始IM撸一个IM》
- ★ 《分布式高性能RPC的基础实操之：千万级用户分布式IM实操- 含简历指导》
- ★ 《亿级用户超高并发秒杀实操- 含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，**2023夏招面试涨薪神器**
- ★ 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- ★ 《第1部曲：超级底层：葵花宝典（高性能秘籍）架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023夏招面试涨薪大神器
- ★ 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- ★ 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- ★ 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- ★ 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- ★ 《架构师实操篇：redis cluster 工业级高可用实操》
- ★ 《架构师实操篇：100W级别QPS日志平台实操》
- ★ 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》
- ★ 《超高并发场景100Wqps三级缓存组件原理和实操》
- ★ 《全链路异步超底层原理和实操：手写hystrix熔断+webflux+Lettuce+Dubbo》
- ★ 《穿透云原生K8S+Jenkins+SpringCloud底层原理和实操》
- ★ 《Golang学习圣经，Go+Java混合 微服务架构 原理与实操》

规划中



左手大数据 (写入简历, 让简历 蓬荜生辉、金光闪闪)

HBASE + Flink + ElasticSearch 原理、架构、真刀实操



右手云原生 (写入简历, 让简历 蓬荜生辉、金光闪闪)

K8S + Devops + ServiceMesh 原理、架构、真刀实操

架构师实操篇: 基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架

架构师实操篇: 千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目

架构师实操篇: 工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目

尼恩JAVA硬核架构班 特色

会员制

提供技术方向指导,
职业生涯指导, 少躺坑, 少弯路

简历指导

有助成功就业、跳槽大厂
挪窝涨薪必备

实操性

项目都是老架构师
在生产上实操过的项目

非水货

老架构师, 不是水货架构师
《Java高并发三部曲》为证

手把手帮扶

让 少部分人 先走向 架构师岗位

2小时简历指导, 传20年内功



架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

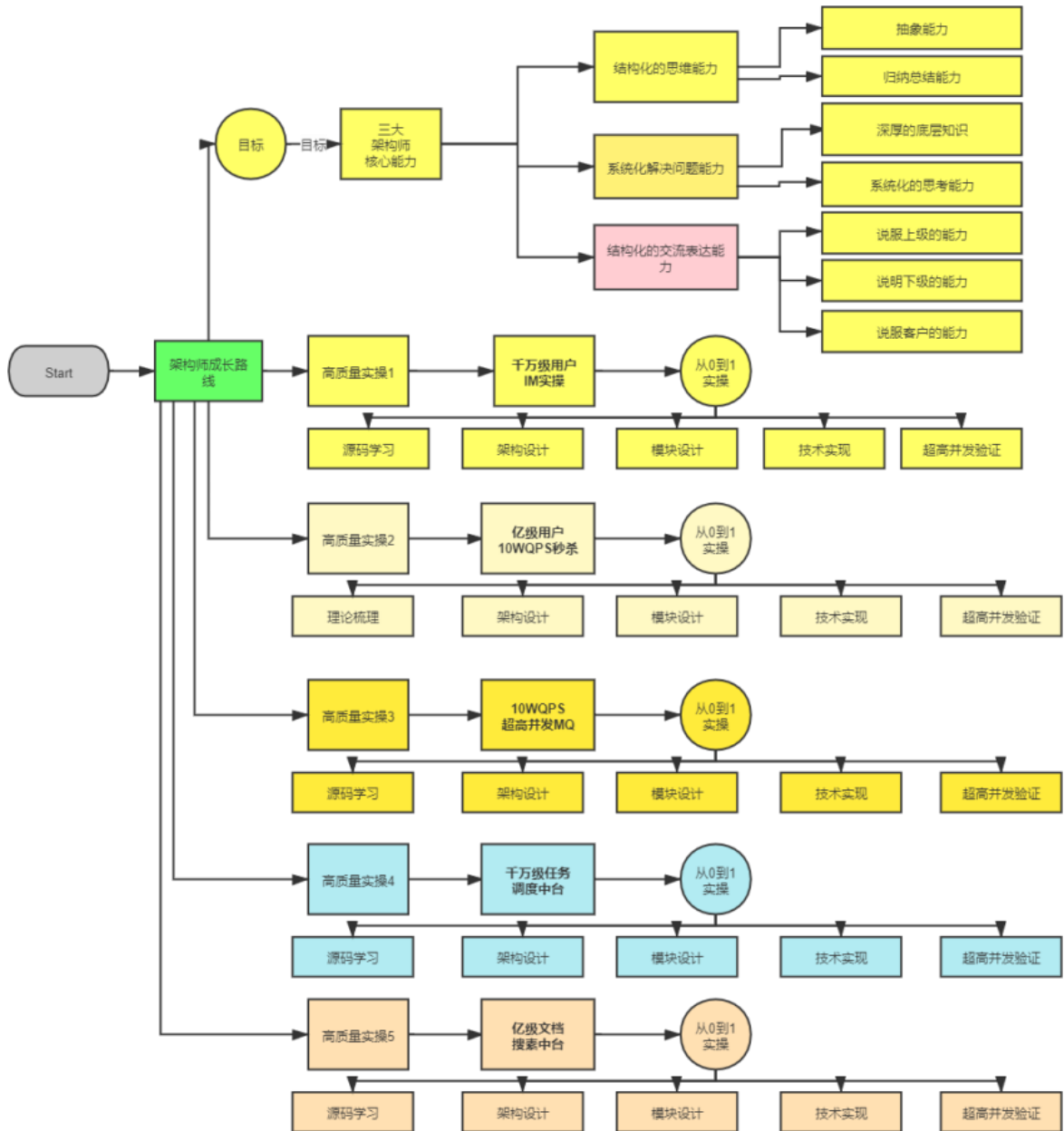
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



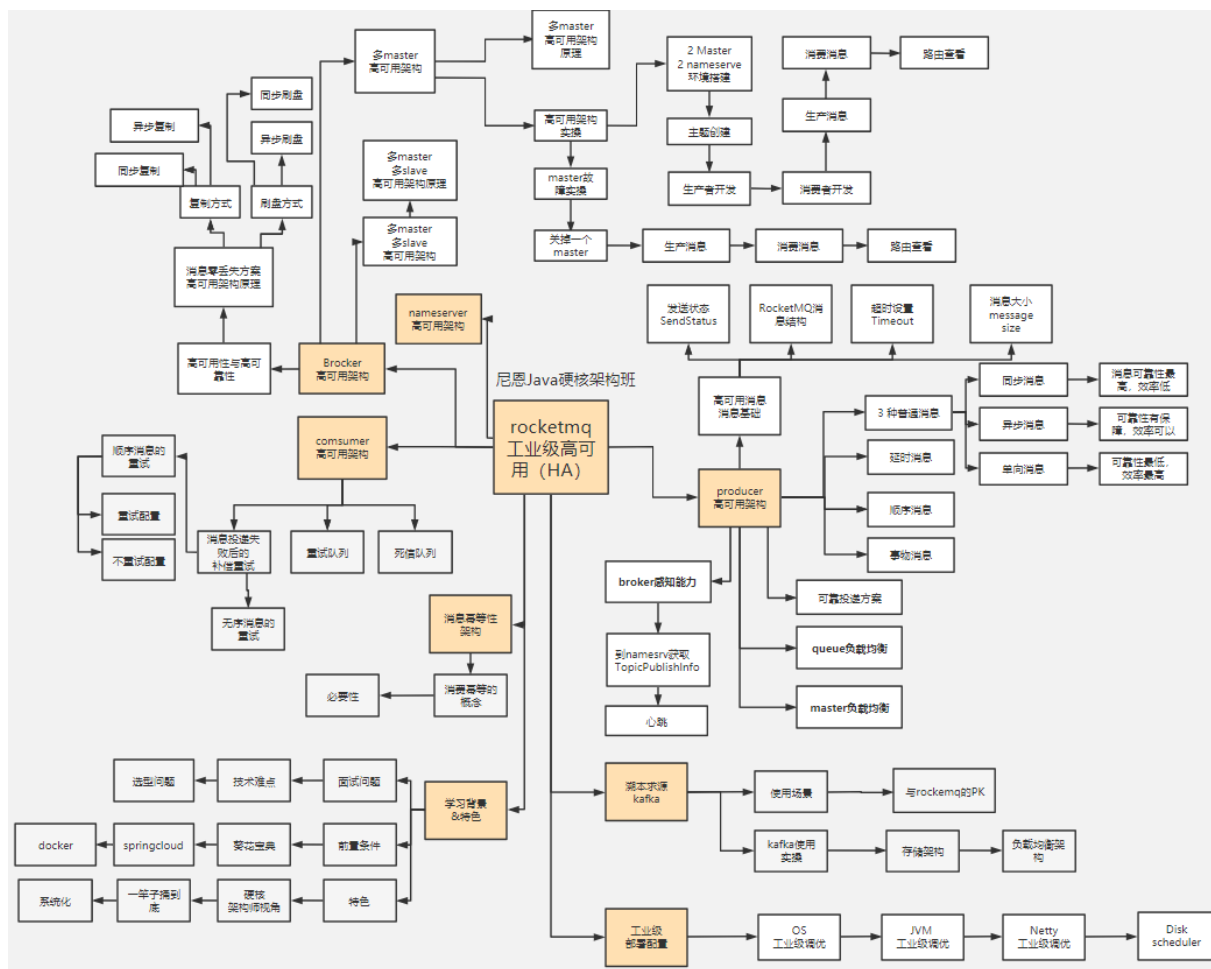
工业级 rocketmq 高可用底层原理和搭建实操，包含：高可用集群的搭建。

1、技术难题：RocketMQ 如何最大限度的保证消息不丢失的呢？RocketMQ 消息如何做到高可靠投递？

2、技术难题：基于消息的分布式事务，核心原理不理解

3、选型难题： kafka or rocketmq ，该娶谁？

下图链接: <https://www.processon.com/view/6178e8ae0e3e7416bde9da19>

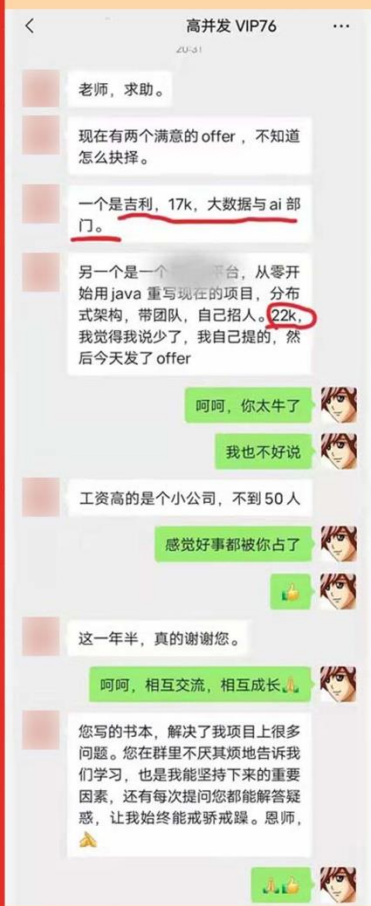


简历优化后的成功涨薪案例（VIP含免费简历优化）

6年专科，2年翻4倍

2年从8K涨到35K

2021年从8K涨到22K



老师，求助。

现在有两个满意的 offer，不知道怎么抉择。

一个是吉利，17k，大数据与 ai 部门。

另一个是一个平台，从零开始用 java 重写现在的项目，分布式架构，带团队，自己招人。22k，我觉得我说少了，我自己提的，然后今天发了 offer

呵呵，你太牛了

我也不好说

工资高的是个小公司，不到 50 人

感觉好事都被你占了

这一年半，真的谢谢您。

呵呵，相互交流，相互成长。

您写的书本，解决了我项目上很多问题。您在群里不厌其烦地告诉我们学习，也是我能坚持下来的重要因素，还有每次提问您都能解答疑惑，让我始终能戒骄戒躁。恩师，

2022年涨到35K



解决了，限制 ip 频率。

谢谢老师

中午12:43

调整到了 35,加上这个月加班费，38

中午12:43

老师，我嗨瑟来了。

晚上8:23

大大的赞

老师你这路子是对的。我就跟着你学习思路和方法，还有教程走的。

我和你一样的兴奋和喜悦

记得咱们去年改简历的时候，还是 10k

这种提升，已经太令人震撼啦

是 8K...

20 年 4 月份转行，就一路跟着你学习

秘诀：
简历指导+ 狠狠卷

6年小伙收60W年薪 一月速提3大offer

4.24号改简历

5.27号报喜

VIP1239 6年

4月24日 晚上19:10

预期的岗位: 60W

预期的岗位: go 和 java 的后端开

4月24日 晚上19:56

4月24日 晚上20:50

6年经验 1239
年薪 60W.
21.7 KB

前几天改ai, 今天改go

谢谢, 我根据这个模式, 再整理一下我简历, 到时候老师再帮我把关

ok

4月27日 下午14:37

老师, 再帮我看看

准备开始投简历了

简历-6年后端开发.pdf
210.1 KB

嗯, 我先对着简历准备些东西, 然后再开始投吧, 反正现在刚刚五一放假

上午8:01

来还愿咯, 3周斩获3个offer, 准备入职了

上午8:05

您太牛啦

简历优化后, 面试机会太多了, 拿完3个offer后, 还有许多公司在流程中的都拒绝了

这几天大动作不断, 联想, 阿里都在裁员, 您太牛啦

还是老师你强, offer中也达到了预期60w

非常不错

现在都上岸有offer就行, 薪酬还能达到预期, 已经超级牛啦

很多人, 连一个面试电话都没有, 崩溃的一塌糊涂

抖音上到处是这种

主要是简历优化后, 感觉如有神助, 每天基本3个面试, 除了字节一面没过, 其它都通过了

恭喜您

谢谢老师

后续有啥问题, 可以找我支援哈

好嘞, 学习圈一直在, 要持续提高自己

好的, 撸起袖子加油卷, 搞技术前途无限好

**秘诀:
简历指导+ 狠狠卷**

被裁后转架构, 逆涨 50% 8年小伙喜提年薪75W

4.16号改简历

5.6号报喜

VIP1236

最近面试了几个一轮游

捞了太多人上岸了

都是你这号

捞我

助力我一个月时间

绞杀 下钻 打破瓶颈

咱们开始不?

好

4月16日 下午15:04

预期岗位: 高级开发、架构

4月16日 下午15:12

预期薪酬: 60W

8-8年高级服务端-0404 - 副本(2).
30.2 KB

微信电脑版

4月16日 下午16:19

**秘诀:
简历指导+ 狠狠卷**

4月16日 下午19:21

通话时长 03:02:25

辛苦老师

尼架, 我决定要去上海了。

拿了几个offer?

两个

上海这个是架构师对吧? 还有一个呢?

还有个广州高级Java, 待遇40w左右, 老板比较喜欢我, 开了很多绿灯, 薪资可以再加, 但我还是想闯闯, 昨天拒绝了。

两年包多少呢?

就之前说的

那都快80个W了

我argue了下, 他们控制内部薪酬平衡有点难办到80, 但已经是标出来的上限了。

都快是高级java的两倍

你撤回了一条消息

75w也非常多了, 在现在的环境下

除开税, 差不多了, 主要是这个方向的潜在价值

关键对你来说, 这个是一个成长机会

是的, 寒意还是有的

您是我的贵人

是! 有个外接大脑就是爽

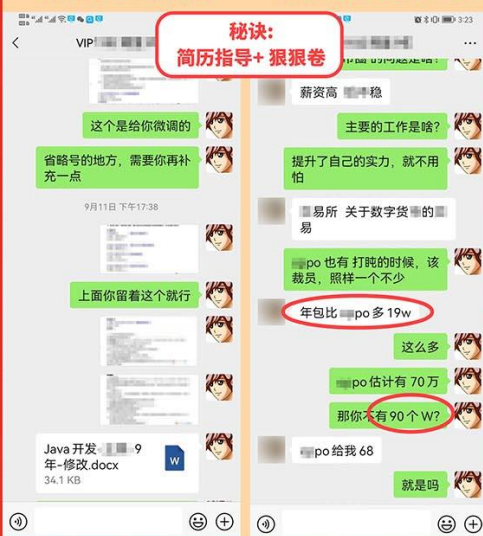
好好卷, 先祝贺您, 拿到年薪75W+

再预祝您, 2年之后, 年薪200W+

谢谢

9年 小伙伴拿到 年薪90W offer

9月11日改简历 11月29日晒offer



小伙8年经验 年薪60w

7月12日改简历 8月10日晒offer



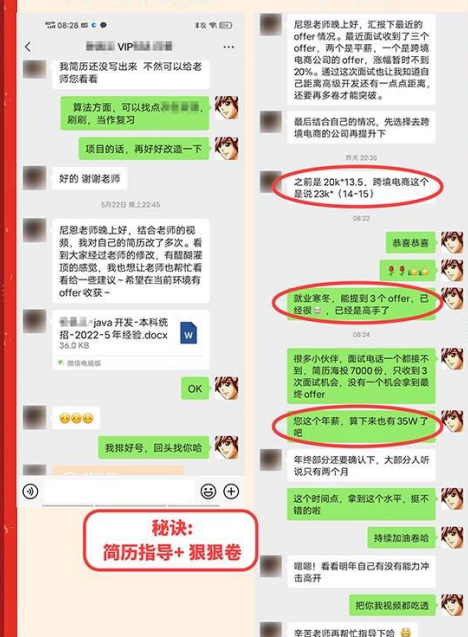
6年小伙伴 年薪40w

9月6日改简历 9月21日晒offer



5年小伙喜提3个offer 年薪 35个W

5月22日改简历 11月29日晒offer



1.5年小伙搞定15K offer 就业寒冬涨100%

5月7日改简历

11月21日晒offer



秘诀:
简历指导+狠狠卷

卷王逆袭成功案例 6年小伙从很少面试机会到 搞定35K*14薪

3月5日改简历

4月11日拿offer



6年 经验小伙伴 喜收25K offer

3月12日改简历

12月1日晒offer



秘诀:
简历指导+狠狠卷

7年经验卷王 薪酬涨30%

7月11日改简历

9月1日晒offer



秘诀:
改简历+狠狠卷

4年经验卷王逆袭 被毕业后，反涨24W

7月改简历 **8月30日晒offer**

**秘诀：
改简历 + 狠狠卷**

这就是你的简历
差得太多啦
ok
总共写了四个项目，最近一年的还没补充上
你是在职，还是离职呢？
离职
原因大概是啥？
项目被终止
方便语音沟通不
ok

是的 感谢你指导，非常重要
15:55
老哥 我八月十号开始找工作，今天已经入职了
现金基本持平，股票+24W
总计涨了多少呢
能涨24W
股票这个吧只能到手了才算
也不错啦
很多小伙伴，面试机会都没有
感谢老哥的指导👍👍，继续跟你卷技术
继续很厉害哈，马上就技术自由啦

小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历 **7月5日晒offer**

**秘诀：
简历指导+ 狠卷3高**

快速看书，就要不求甚解，把目录和场景大概一下，然后重点的地方，用划的地方，再去回顾
5月29日 上午10:58
尼恩 我被"毕业"了
这周末或下周找你改一下简历
毕业没有关系
ok，发我吧
it行业，就来跑去，太频繁啦
嗯，其实有点心理准备
5月29日 上午10:49
简历指导
5月29日 上午10:52
不太会写简历

尼恩 我拿到半职的offer了
10:54
涨20%，2家要多，结果人家都不还价的
看起来半职不差钱呀
超过了8000没
平均算下来
7000多
好的
有啥面试的心得吗
可以分享给其他小伙伴的
1 面试前要多准备，2 面试时不要紧张，3 面试后要感谢面试官

卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历 **4月15日晒offer**

**面试法宝：
改简历 + 实操**

尼恩老师，新年好！
能帮忙修改下简历吗？
金三银四准备挑了
可以的
java开发-6年-简历-
340.2 KB
拜托了，尼恩。希望能拿25k回来给你报喜
2月10日 上午9:57
好，我加一下
还有吗？
2月10日 上午10:10

尼恩，决意面offer了
截图是我目前认知能写出来的评分了，麻烦帮我参考下
选择大于努力，尼恩助我上岸
这么多offer，我看看哈
都是尼恩指点有方👍👍，本来还有个新能源汽车的，35W给拒了，主要太远了
跟着尼恩老师的时间太短了，目前实力也只能到这儿了
这边有个大数据的，感觉也不错

卷王逆袭成功案例 6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历 **5月17日晒offer**

**涨薪法宝：
改简历 + 狠狠卷**

Java开发工程师_...
dock
52.5 KB
微信语音
你看到我给你改的
好的呀
4月14日 晚上22:23
麻烦大佬了
这个你自己别哈
不对的，你自己别
那我照着这个改一下库存系统呀
一个简历，...
这么漂亮的简历，涨50%，已经没啥问题
只要准备好，不出大批量，基本没问题啦

保密押金收起来哈，你的offer最高涨了9k，多返现100
好的
谢谢大佬
后面继续跟尼恩卷哈，感觉卷的时间越长，...
尼恩，...
我准备这一年的时间都看呢
加油卷哈
感觉自己学的不太透彻了
嗯嗯
跟着大佬一起
我周围好几个年薪百万的，都是这...

卷王逆袭成功案例

5年经验小伙收2个offer 最高涨薪8k，年薪42W

5月9日改简历

5月30日晒offer

秘诀:
简历指导+ 狠卷3高

以此为样
大家狠狠卷
打造最卷IT社群

卷王逆袭成功案例

非全日制 6年经验卷王 喜提3个Offer，年包30W

5月9日改简历

5月18日晒offer

面试法宝:
改简历+ 狠狠卷

卷王逆袭成功案例

寒五冻六之际卷王大逆袭 收3大offer，涨30%

5月17日改简历

5月27日晒offer

秘诀:
简历指导+ 狠卷3高

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历

5月12日晒offer

涨薪法宝:
改简历+ 狠狠卷

4年小伙喜收百度、Boss直聘等N个顶级Offer
最高涨幅100%

6月27日改简历

9月19日晒offer

秘诀:
改简历+狠狠卷

有个offer选择问题

boss直聘和小满之间, boss那

还有好几个offer, 还有一个养

总体就想看 boss 和小满之间

了

boss 比小满年包多 7w 以上,

我这涨幅都接近百分之百了

卷王逆袭成功案例

4年卷王入收2个offer, 涨50%

3月23日改简历

5月12日晒offer

offer 决策圈

涨薪法宝:
改简历+狠狠卷

涨50%的样子

小伙大三暑期很焦虑
跟着尼恩卷一年
校招斩获顶级央企Offer

去年5月19日加入VIP群

今年7月5日晒offer

秘诀:
狠狠卷书+视频

尼恩老师
我校招去华润电力控股有限公司了
跟着你卷了一年 大学顺便拿了几个国奖
不错不错, 这是央企

小伙高中学历
薪酬涨120%

5月6日改简历

7月22日晒offer

秘诀:
改简历+狠狠卷

卷王逆袭成功案例

非全日制卷王 面试3家 收2个offer 涨薪30%

4月13日改简历

4月21日晒offer

面试法宝:
改简历 + 面试题

5年卷王喜收2大Offer

最高涨5K

5月19日改简历

9月13日晒offer

秘诀:
改简历 + 狠狠卷

卷王逆袭成功案例

3年经验卷王, 涨60%

4月16日改简历

5月11日晒offer

涨薪法宝:
改简历 + 狠狠卷

卷王逆袭成功案例

双非二本小伙春招大翻身 喜提9大offer

2月22日改简历

4月13日晒offer

面试法宝:
改简历 + IM实操

1	公司	部门	岗位	薪资结构	总包
2	小米	数字产品部	java后端开发	18.5k+14.5k+5k+2000/月+500/月+500/月	22.4w
3	小米	交易研发部	java后端开发	18k+14.5k+5k+2000/月+500/月	22.4w
4	小米	待定	java游戏开发	15k+15k+餐补+1000/月	>22.5w
5	小米	待定	java后端开发	11k+13k	14.2w
6	小米	待定	java后端开发	9k	>16.8w
7	小米	待定	java后端开发	9k+餐补+社保+公积金	17w
8	小米	待定	java后端开发	10k+餐补+社保+公积金	17w
9	小米	待定	java后端开发	10k+餐补+社保+公积金	17w

9大offer 最高年薪30万

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>